GDT & LDT

Xeno Kovah – 2021
xeno@darkmentor.com

All materials is licensed under a Creative Commons “Share Alike” license.

- http://creativecommons.org/licenses/by-sa/4.0/

You are free:

to Share — to copy, distribute and transmit the work

to Remix — to adapt the work

Under the following conditions:

Attribution — You must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).

Share Alike — If you alter, transform, or build upon this work, you may distribute the resulting work only under the same, similar or a compatible license.

Attribution condition: You must indicate that derivative work
"Is derived from Xeno Kovah's 'Architecture 2001: x86-64 OS Internals’ class, available at https://ost.fyi"

GDT & LDT

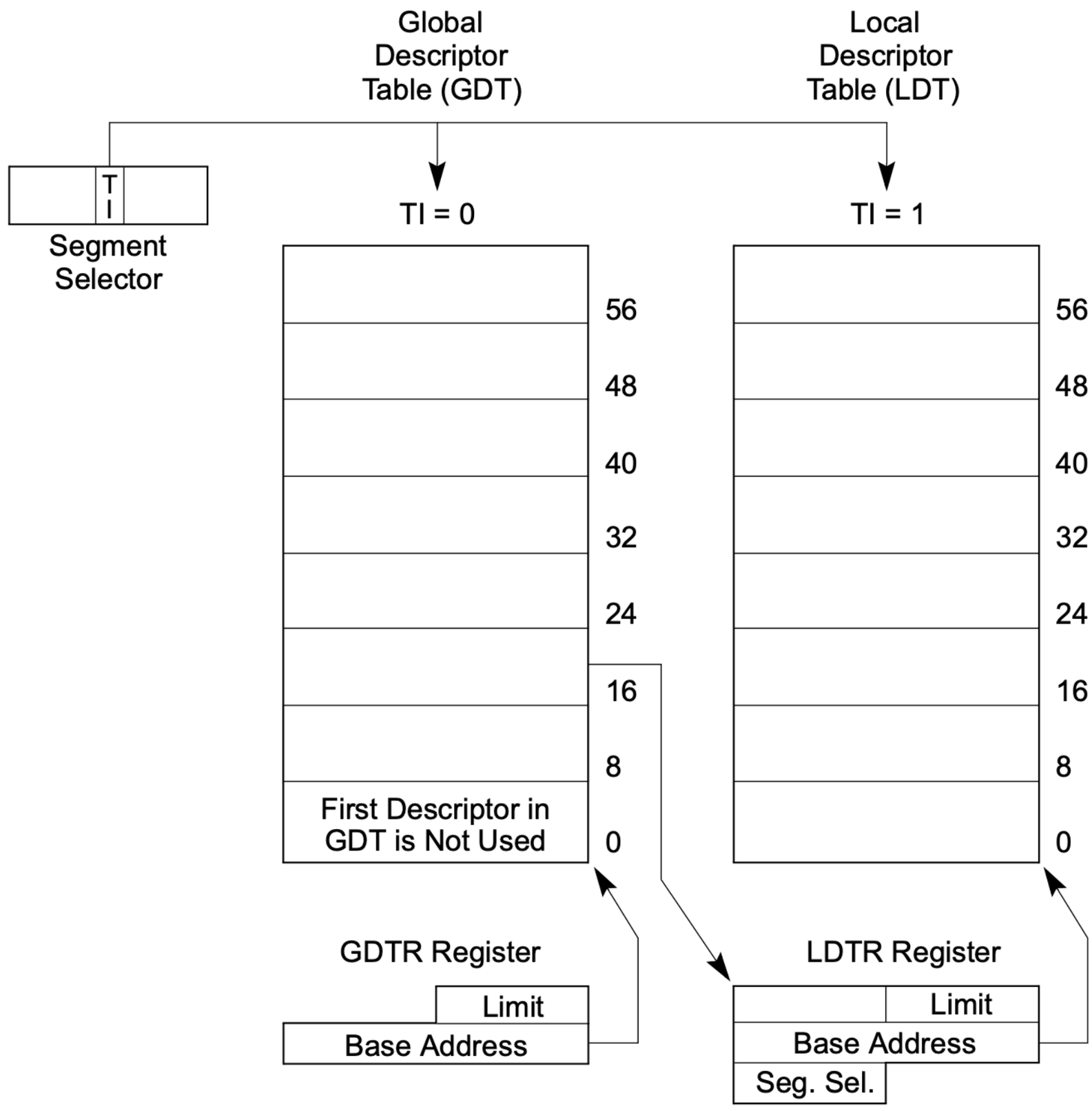
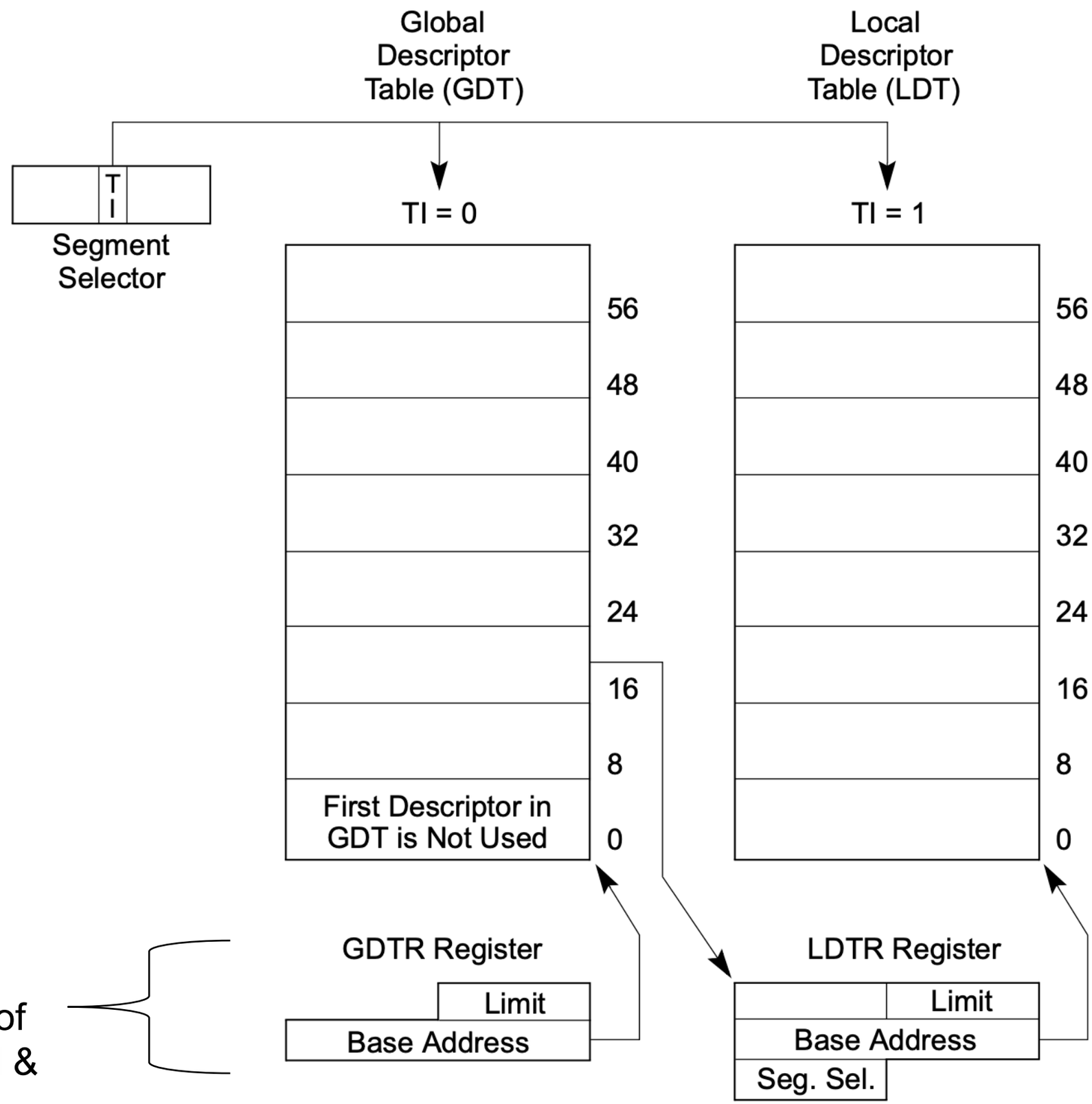


Figure 3-10. Global and Local Descriptor Tables

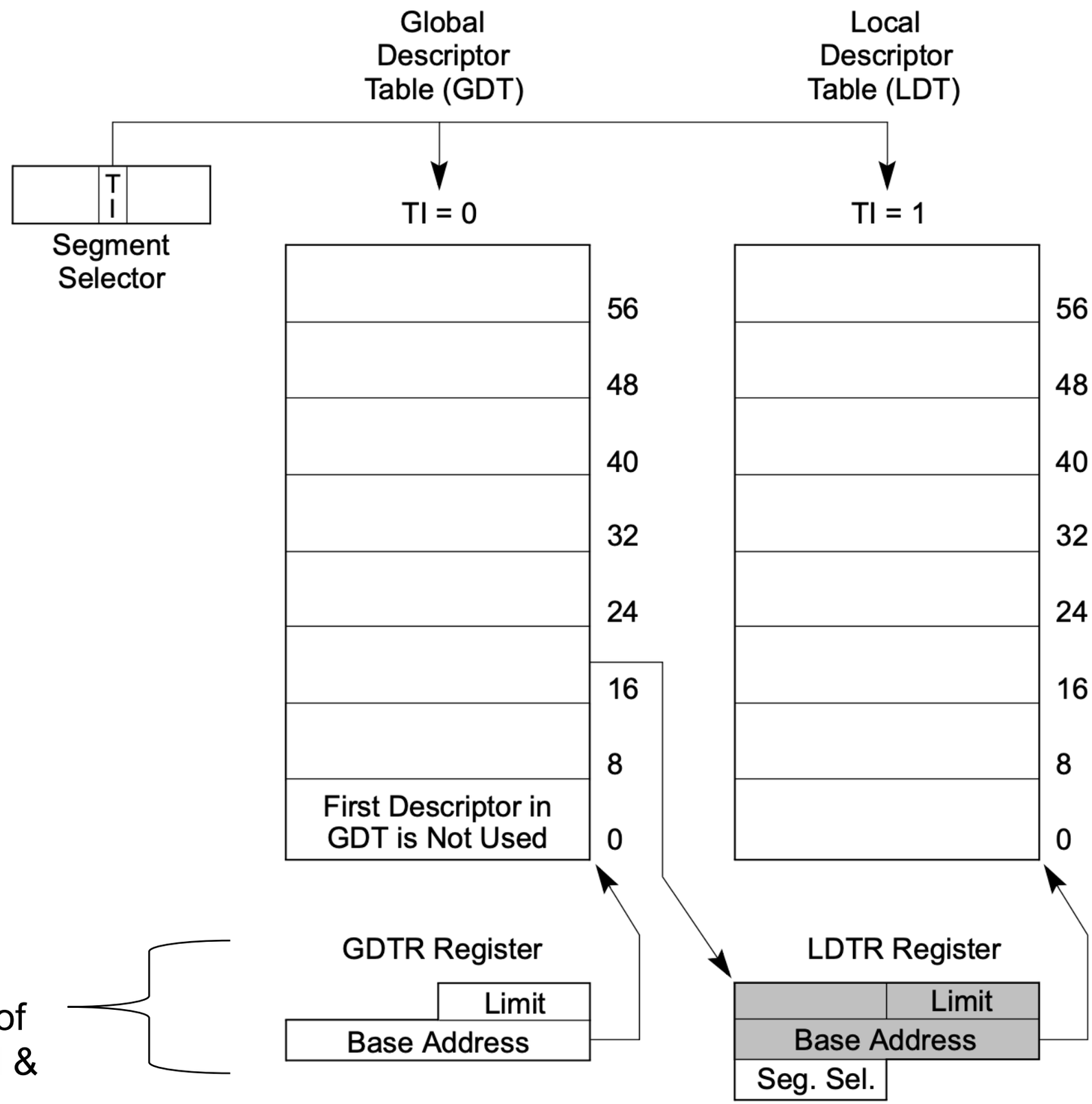
GDT & LDT



Special registers point to the base of the tables in RAM & specify their size

Figure 3-10. Global and Local Descriptor Tables

GDT & LDT



Special registers point to the base of the tables in RAM & specify their size

Figure 3-10. Global and Local Descriptor Tables

GDT & LDT

All entries in these tables are “Segment Descriptor” structures

Special registers point to the base of the tables in RAM & specify their size

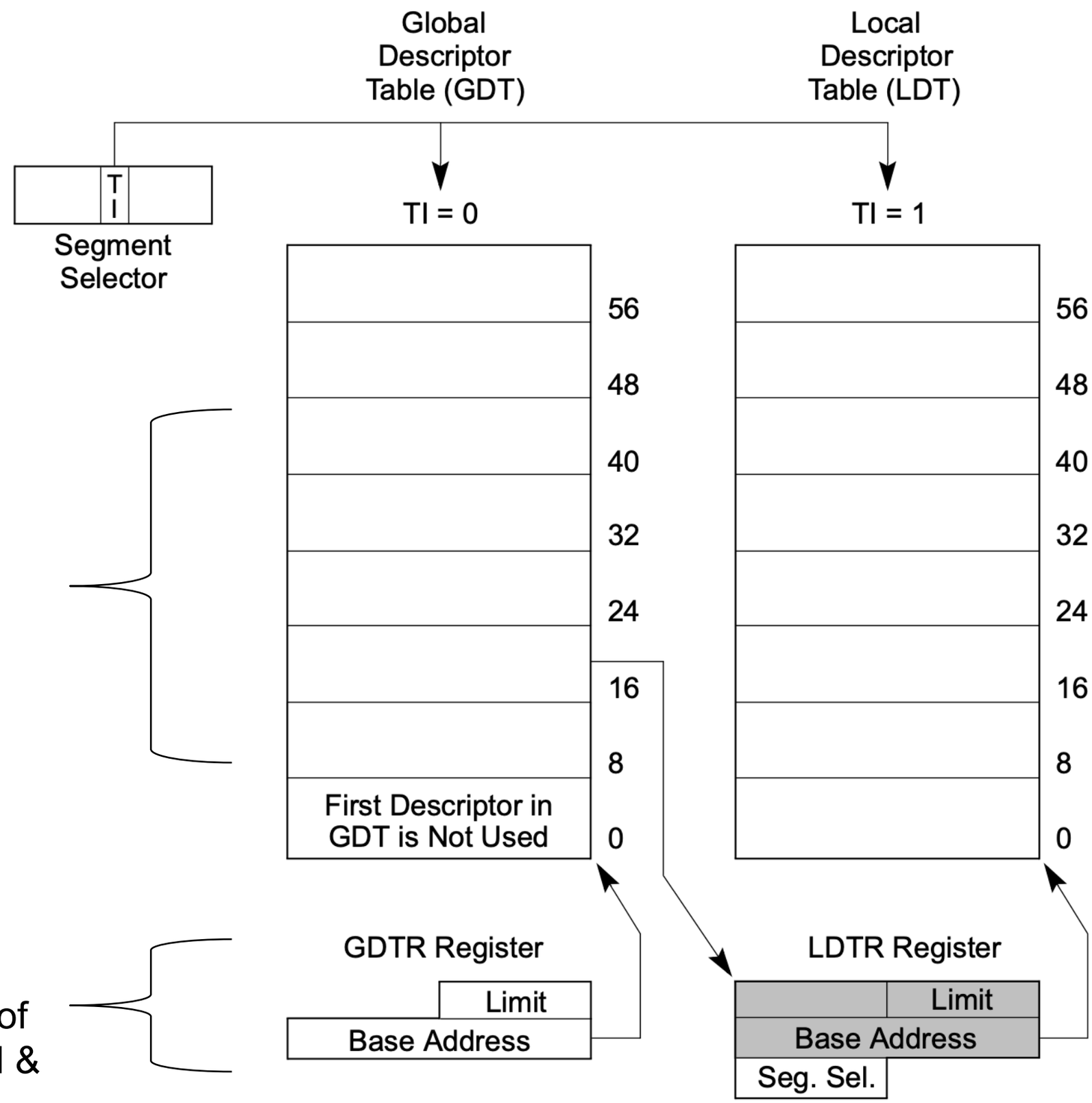
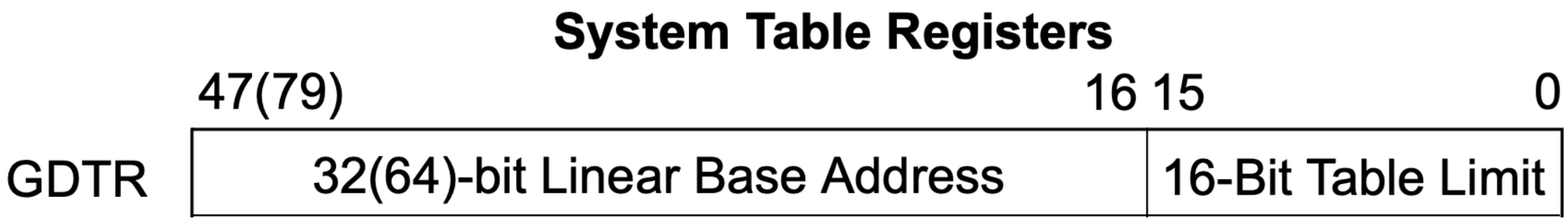


Figure 3-10. Global and Local Descriptor Tables

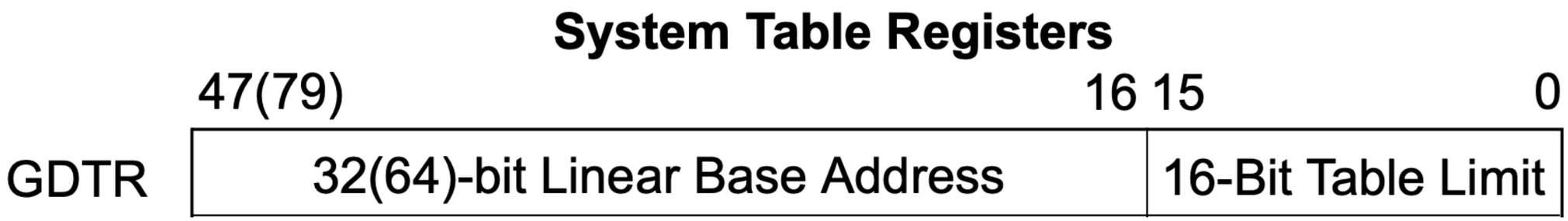
Global Descriptor Table *Register* (GDTR)



From Vol 3, Figure 2-6

- The upper 64 bits ("base address") of the register specifies the *linear* address where the GDT is stored
- The lower 16 bits ("table limit") specify the size of the table *in bytes* (with base+limit specifying the address of the last valid/included byte)
- Special instructions used to load a value into the register or store the value out to memory
  - LGDT - Load 10 bytes from memory into GDTR
  - SGDT - Store 10 bytes of GDTR to memory

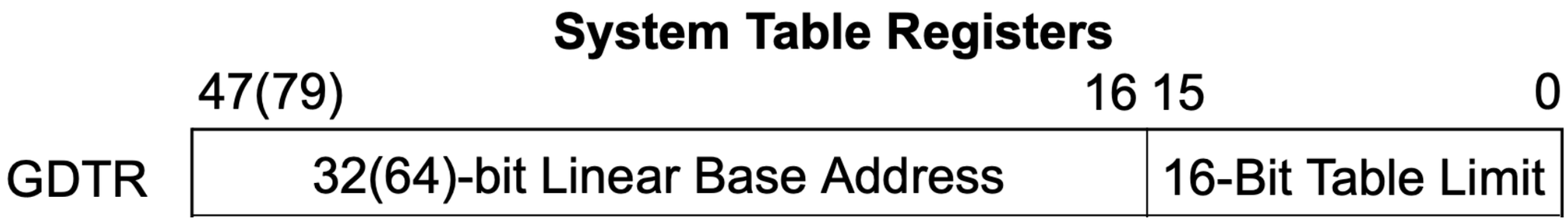
Global Descriptor Table *Register* (GDTR)



From Vol 3, Figure 2-6

- The upper 64 bits ("base address") of the register specifies the *linear* address where the GDT is stored
- The lower 16 bits ("table limit") specify the size of the table *in bytes* (with base+limit specifying the address of the last valid/included byte)
- Special instructions used to load a value into the register or store the value out to memory
  - LGDT - Load 10 bytes from memory into GDTR
  - SGDT - Store 10 bytes of GDTR to memory

Global Descriptor Table *Register* (GDTR)



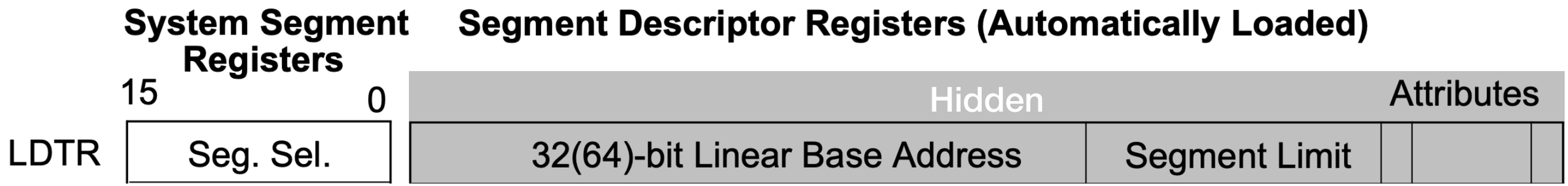
From Vol 3, Figure 2-6

- The upper 64 bits ("base address") of the register specifies the *linear* address where the GDT is stored
- The lower 16 bits ("table limit") specify the size of the table *in bytes* (with base+limit specifying the address of the last valid/included byte)
- Special instructions used to load a value into the register or store the value out to memory
  - 9 LGDT - Load 10 bytes from memory into GDTR
  - 10 SGDT - Store 10 bytes of GDTR to memory

Lab: Go look at the GDTR

- Break into the debugee from the debugger
- Scroll all the way down your register window to find the "gdtr" and "gdtl" registers
- "gdtl" is just a WinDbg-ism for the bottom 16 bits of GDTR which specify the size (in bytes) of the overall GDT. It's not an actual register!
- What's the base address of the GDT?
- What's the size of the GDT?

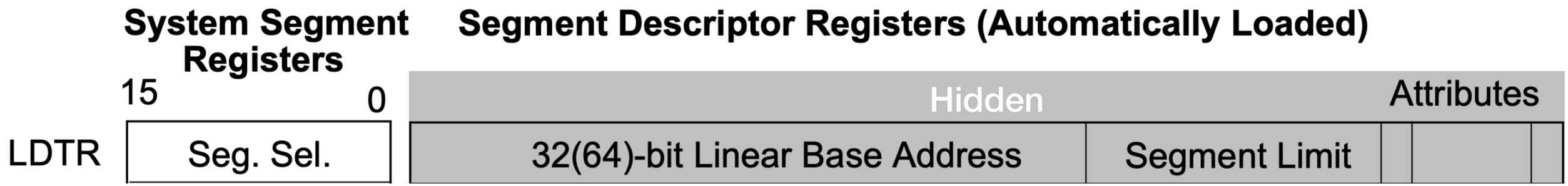
Local Descriptor Table *Register* (LDTR)



From Vol 3, Figure 2-6

- Like the 16-bit segment registers, the 16-bit LDTR has a visible part, the segment selector, and a hidden part, the cached segment info which specifies the size of the LDT
  - The selector's Table Indicator (T) bit must be set to 0 to specify that it's selecting from the GDT, not from itself ;)
- Special instructions used to load a value into the register or store the value out to memory
  - LLDT - Load 16 bit segment selector into LDTR
  - SLDT - Store 16 bit segment selector of LDTR to memory

Local Descriptor Table *Register* (LDTR)

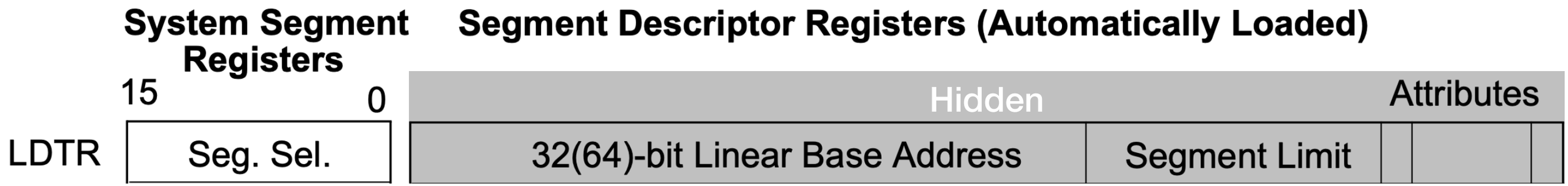


From Vol 3, Figure 2-6

- Like the 16-bit segment registers, the 16-bit LDTR has a visible part, the segment selector, and a hidden part, the cached segment info which specifies the size of the LDT
  - The selector's Table Indicator (T) bit must be set to 0 to specify that it's selecting from the GDT, not from itself ;)
- Special instructions used to load a value into the register or store the value out to memory
  - LLDT - Load 16 bit segment selector into LDTR
  - SLDT - Store 16 bit segment selector of LDTR to memory

11

Local Descriptor Table *Register* (LDTR)



From Vol 3, Figure 2-6

- Like the 16-bit segment registers, the 16-bit LDTR has a visible part, the segment selector, and a hidden part, the cached segment info which specifies the size of the LDT
  - The selector's Table Indicator (T) bit must be set to 0 to specify that it's selecting from the GDT, not from itself ;)
- Special instructions used to load a value into the register or store the value out to memory
  - 11 LLDT - Load 16 bit segment selector into LDTR
  - 12 SLDT - Store 16 bit segment selector of LDTR to memory

Lab: Go look at the LDTR

- Break into the debugee from the debugger
- Scroll all the way down your register window to find the "ldtr" registers.
- What index in the GDT is used for the LDT?

GDT & LDT

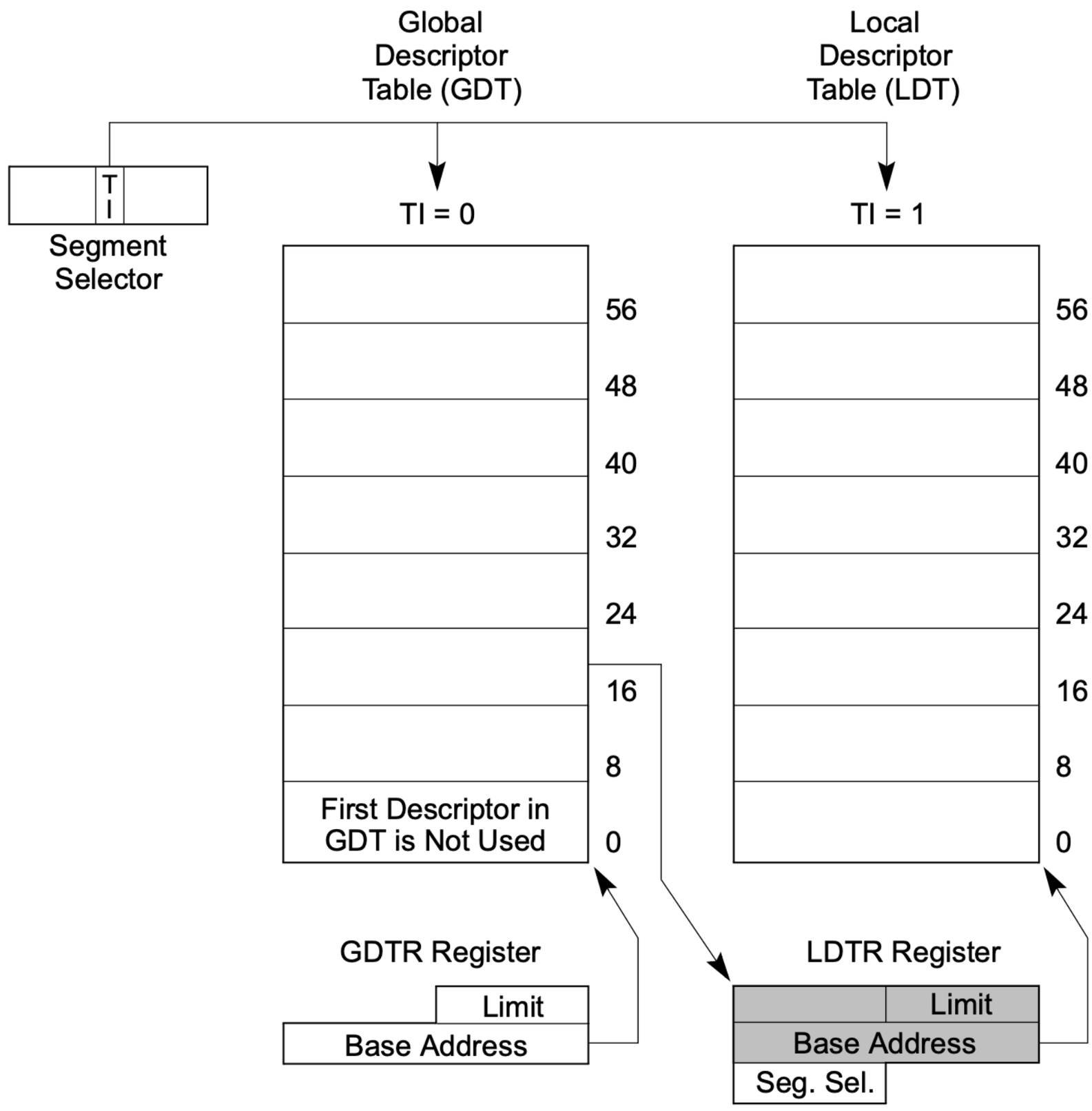


Figure 3-10. Global and Local Descriptor Tables

GDT & LDT

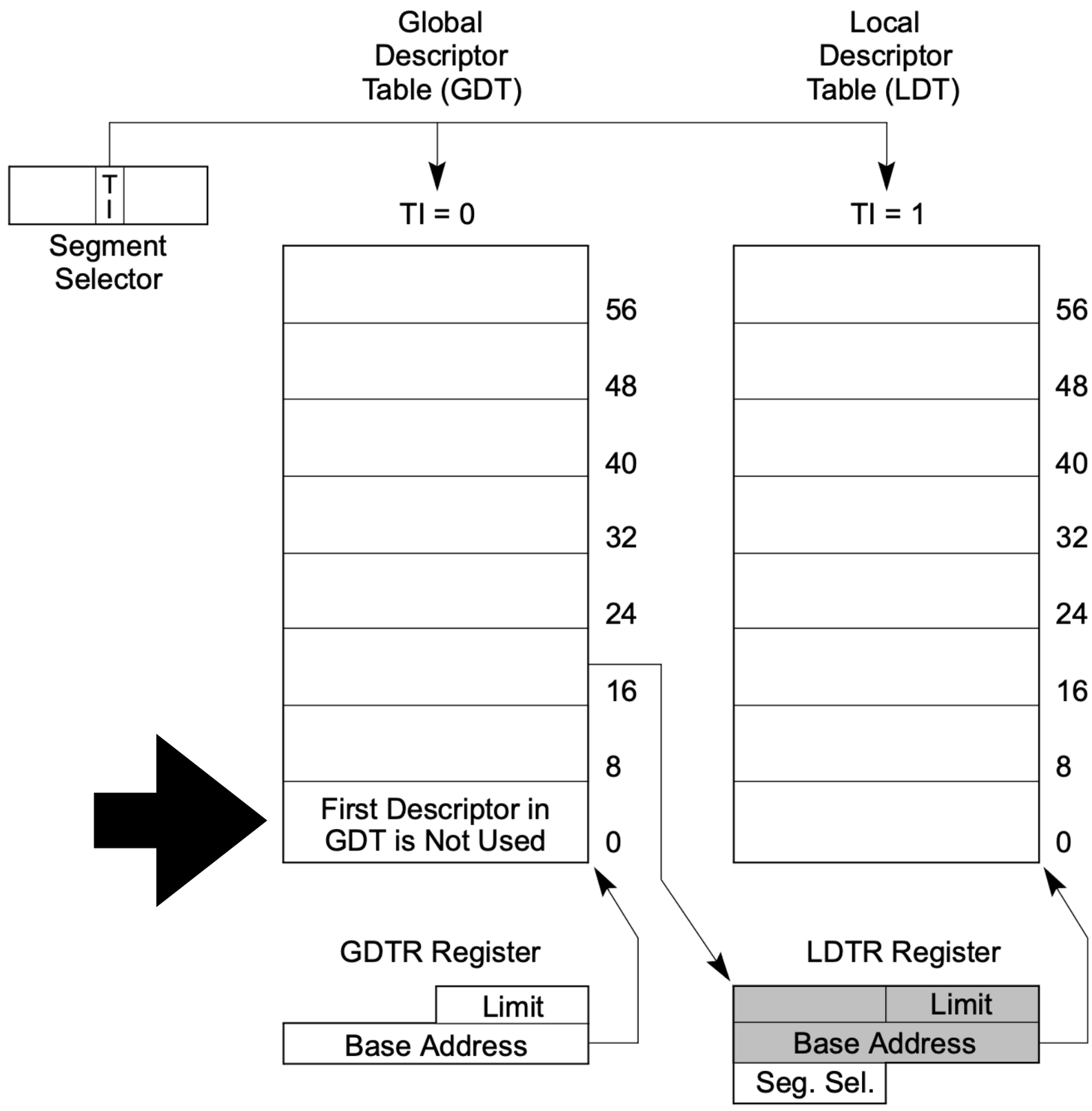


Figure 3-10. Global and Local Descriptor Tables

GDT & LDT

The *Segment Descriptor* doesn't *point to* the LDTR register! The register's *Segment Selector* "points to" the *Segment Descriptor*! (And causes it to get loaded into the hidden area of the register)

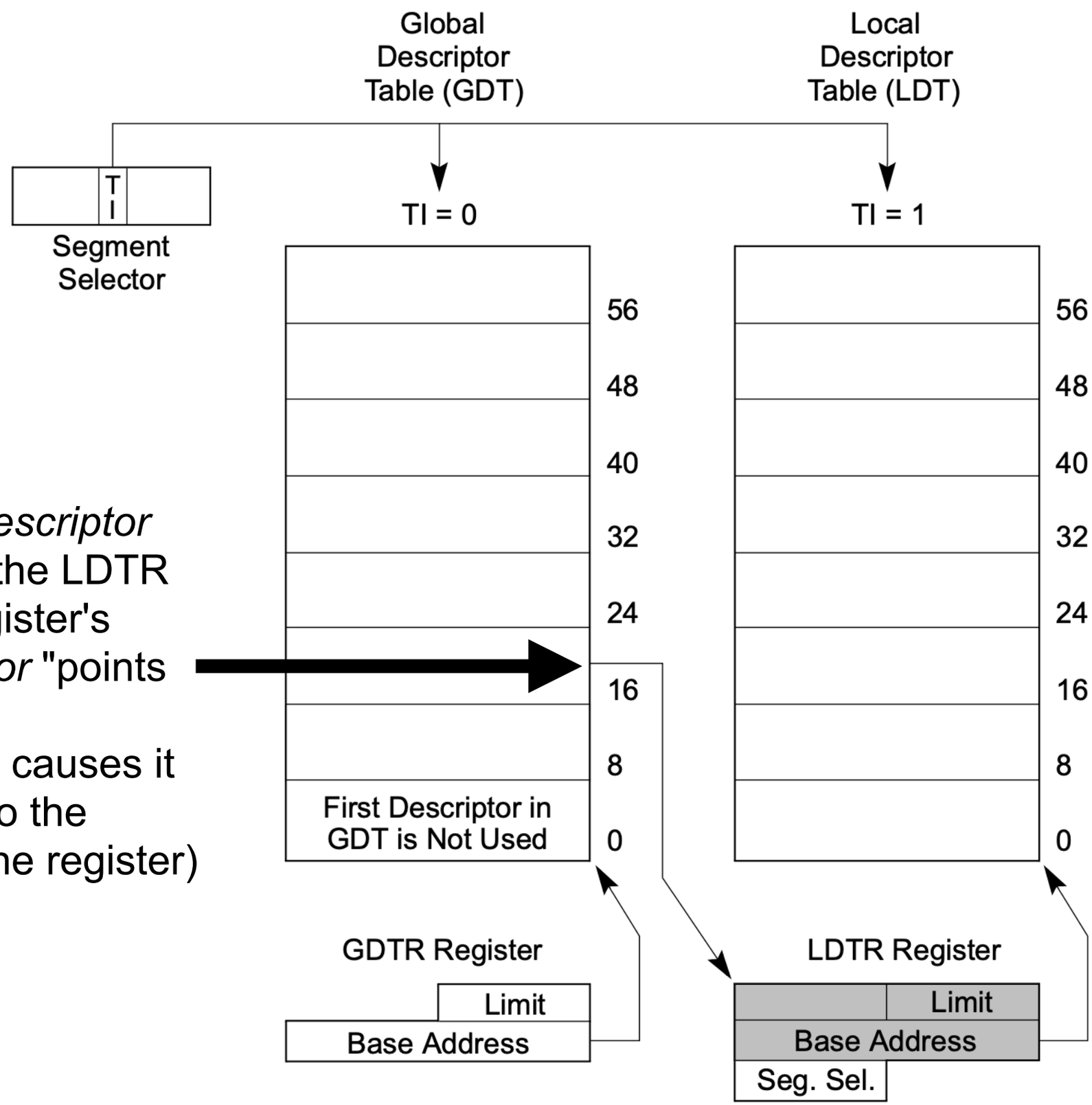


Figure 3-10. Global and Local Descriptor Tables

GDT & LDT

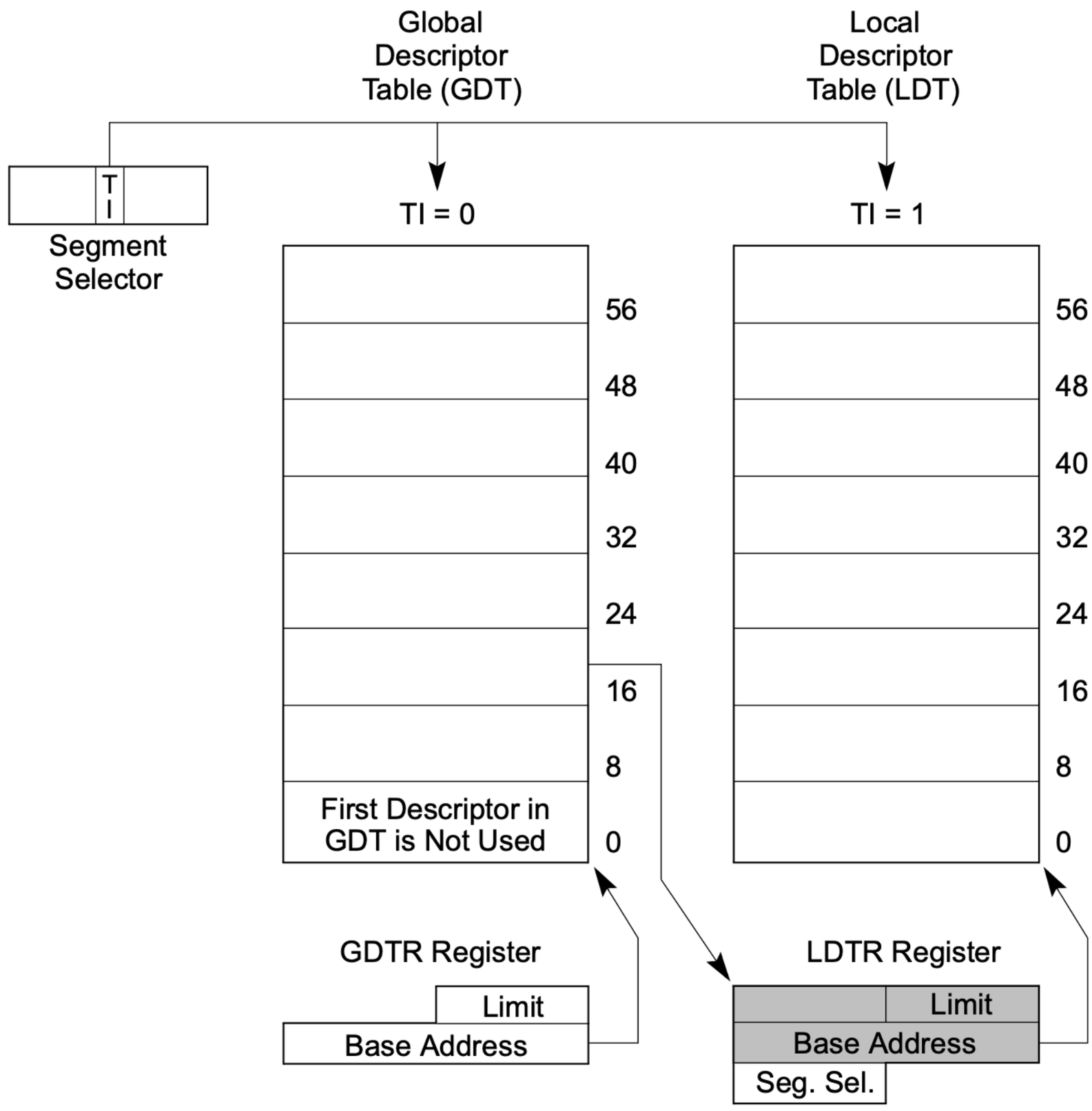
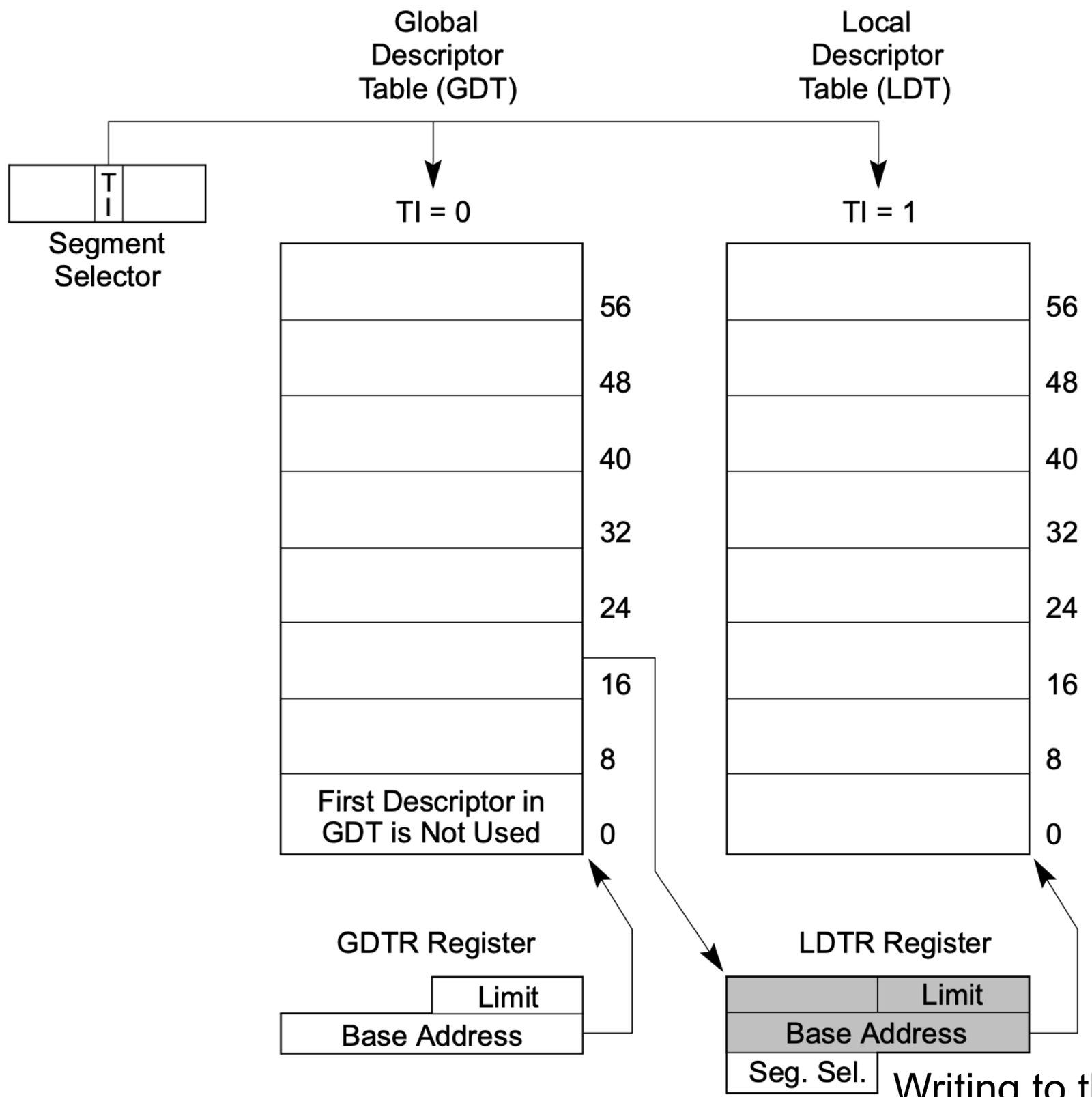


Figure 3-10. Global and Local Descriptor Tables

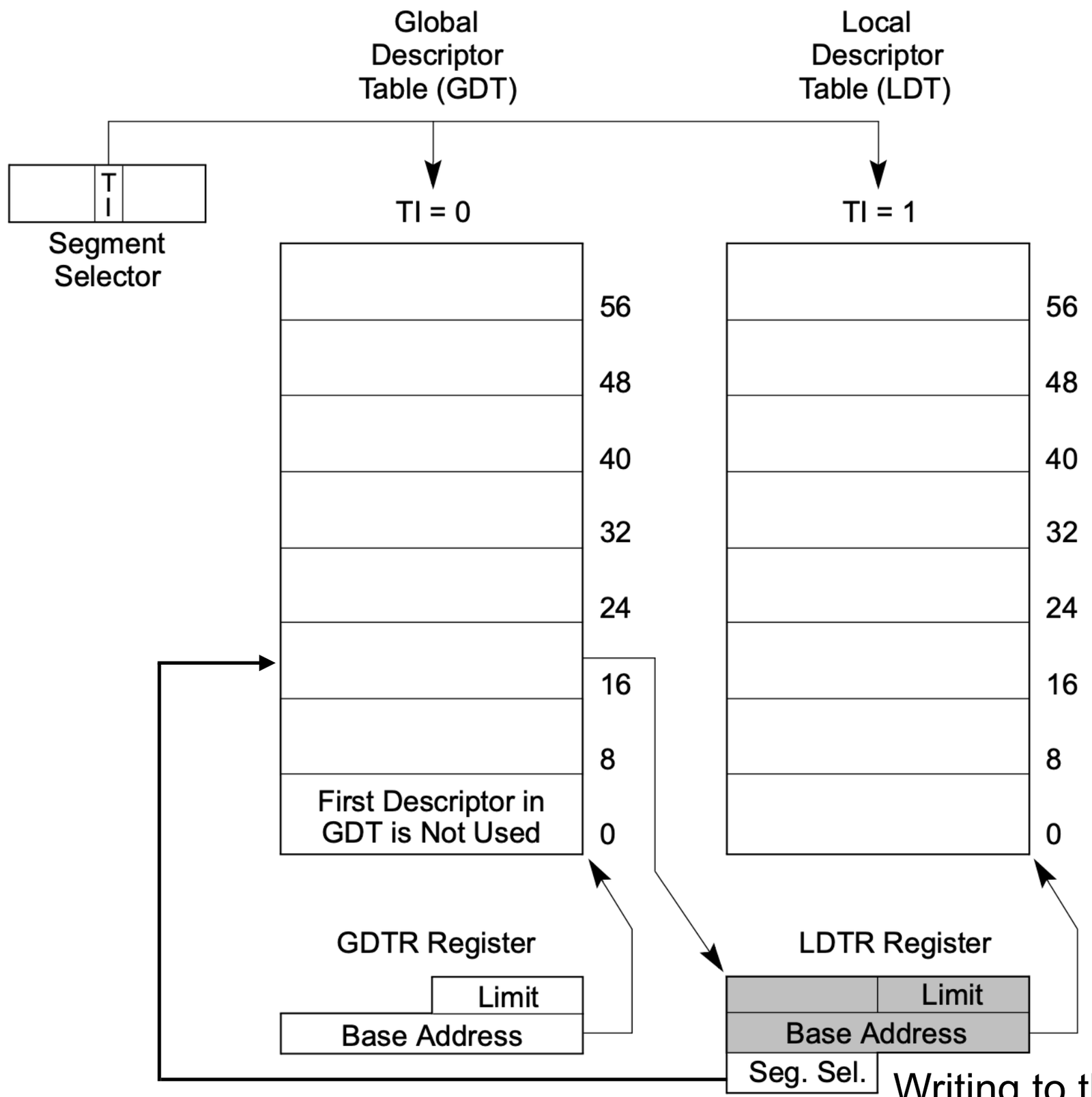
GDT & LDT



Writing to the LDTR segment selector...

Figure 3-10. Global and Local Descriptor Tat

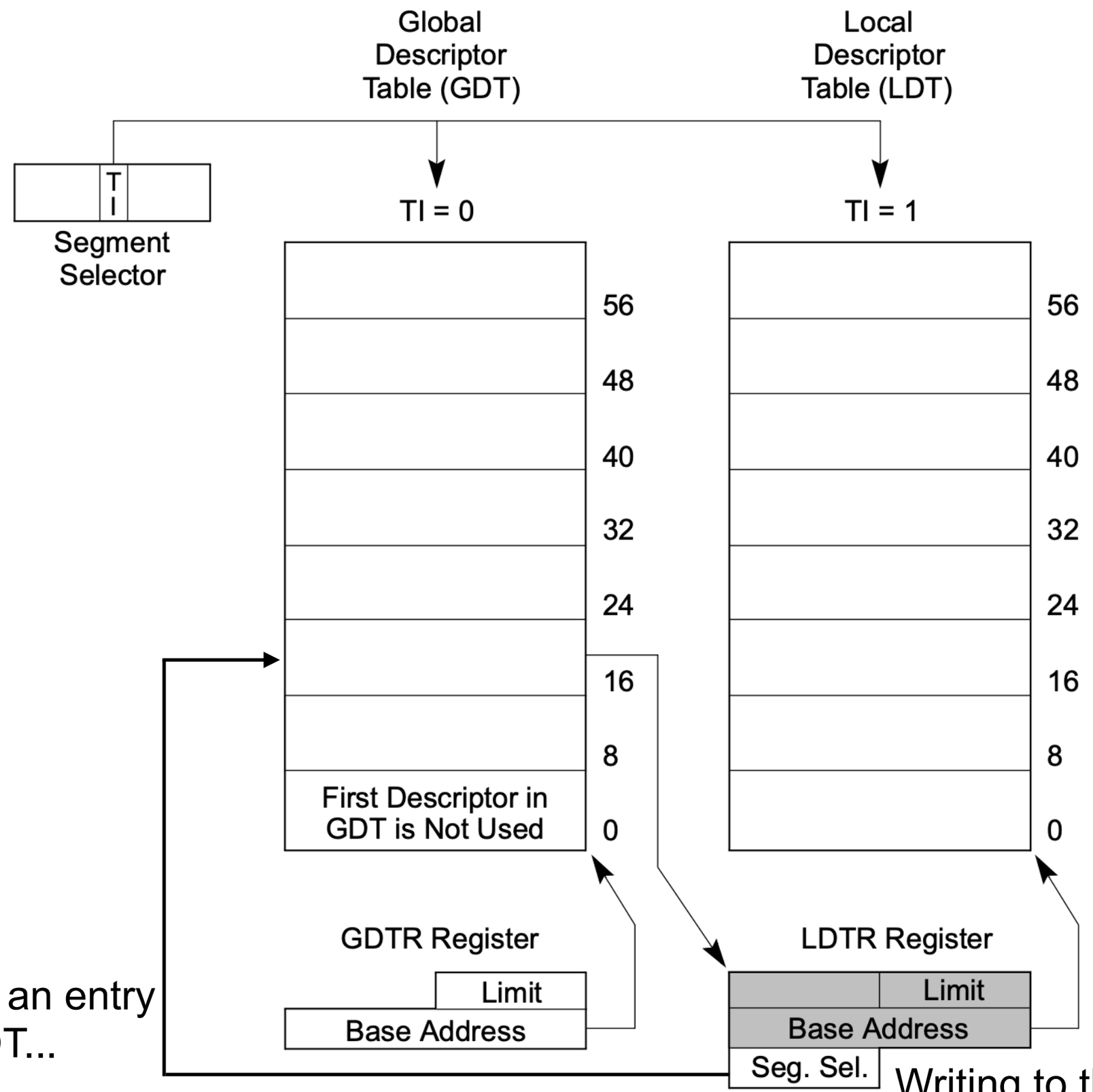
GDT & LDT



Writing to the LDTR segment selector...

Figure 3-10. Global and Local Descriptor Tat

GDT & LDT

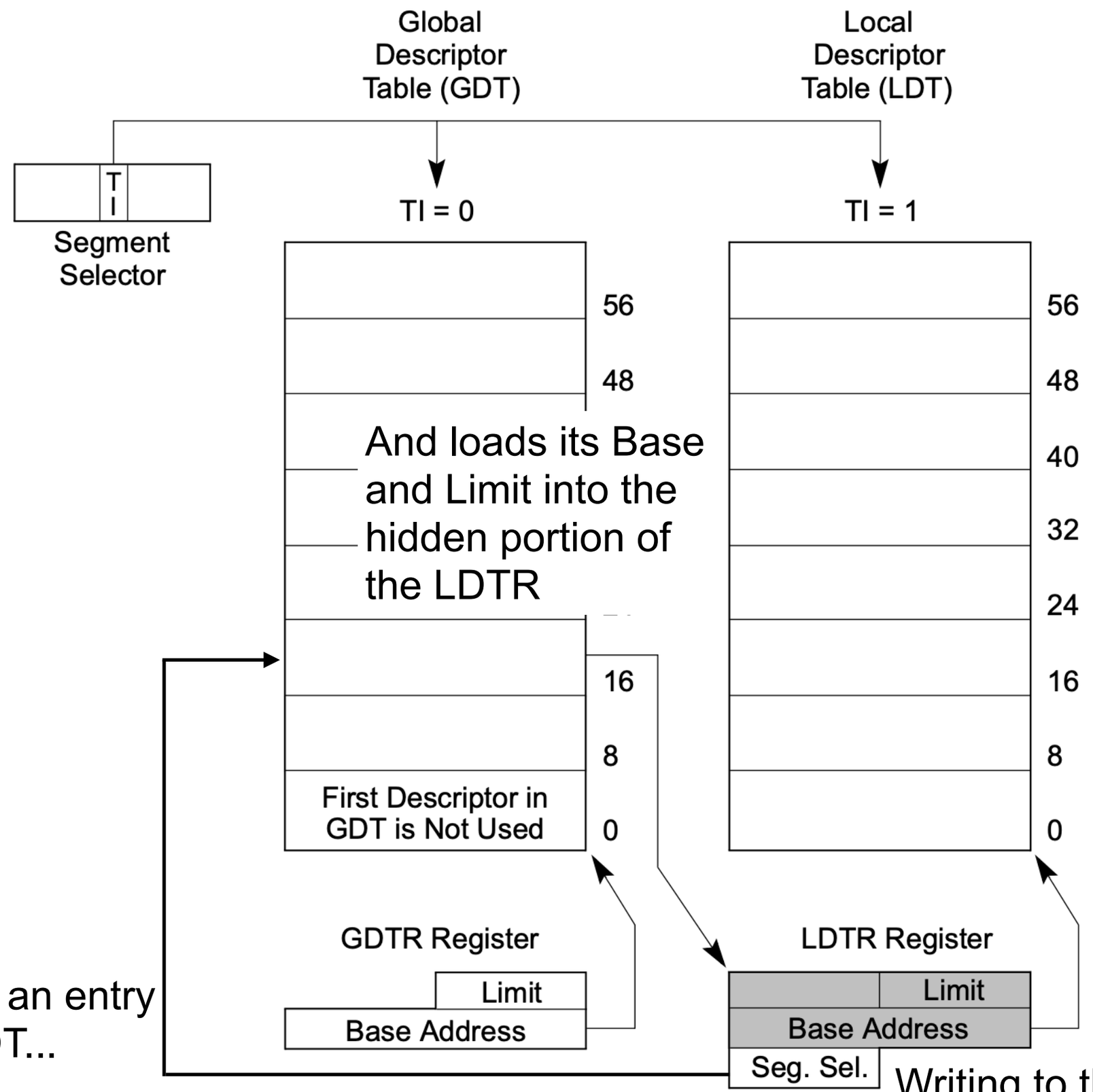


Looks up an entry in the GDT...

Writing to the LDTR segment selector...

Figure 3-10. Global and Local Descriptor Tat

GDT & LDT

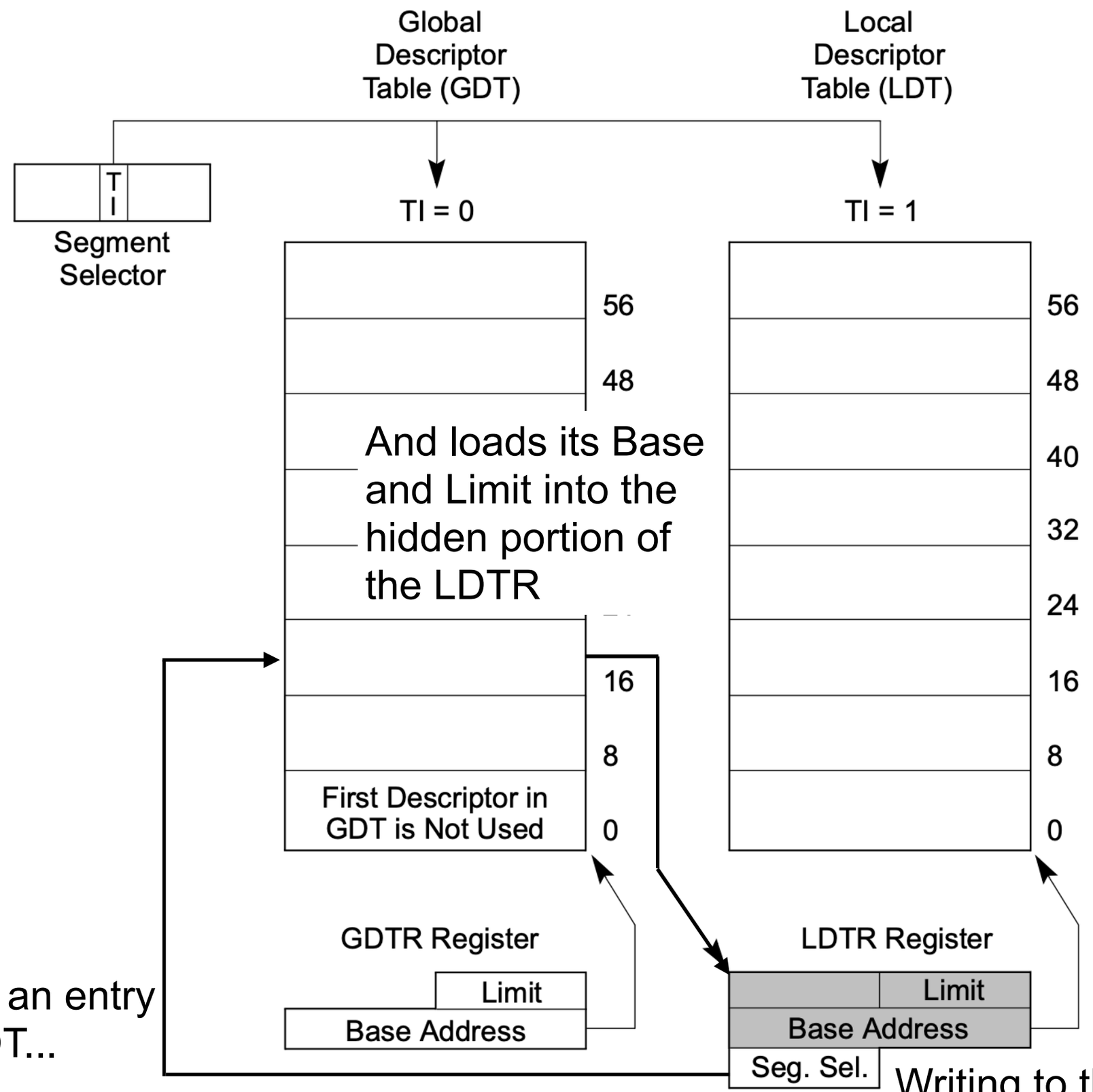


Looks up an entry in the GDT...

Writing to the LDTR segment selector...

Figure 3-10. Global and Local Descriptor Tat

GDT & LDT



Looks up an entry in the GDT...

Writing to the LDTR segment selector...

Figure 3-10. Global and Local Descriptor Tat

GDT & LDT

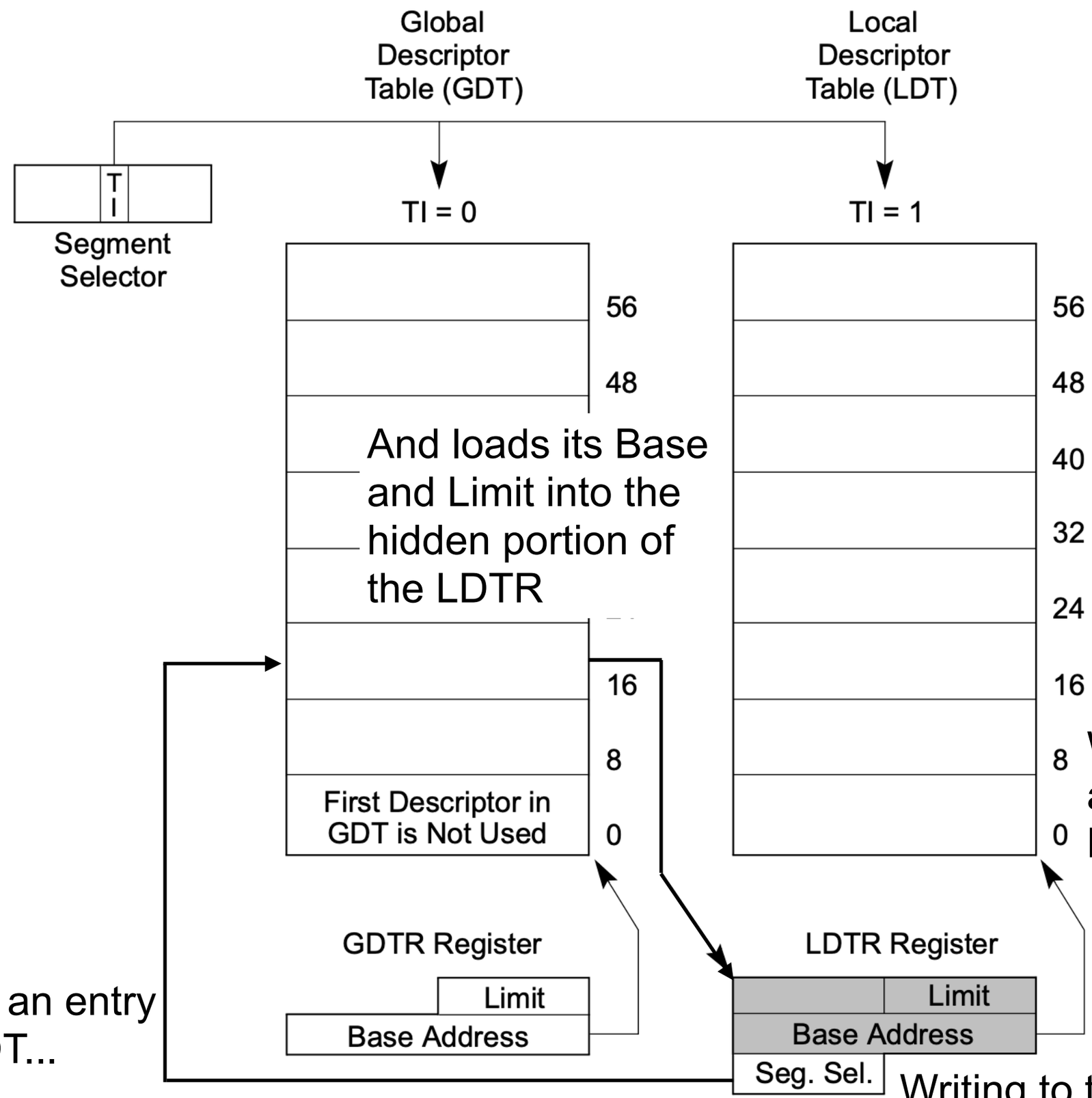
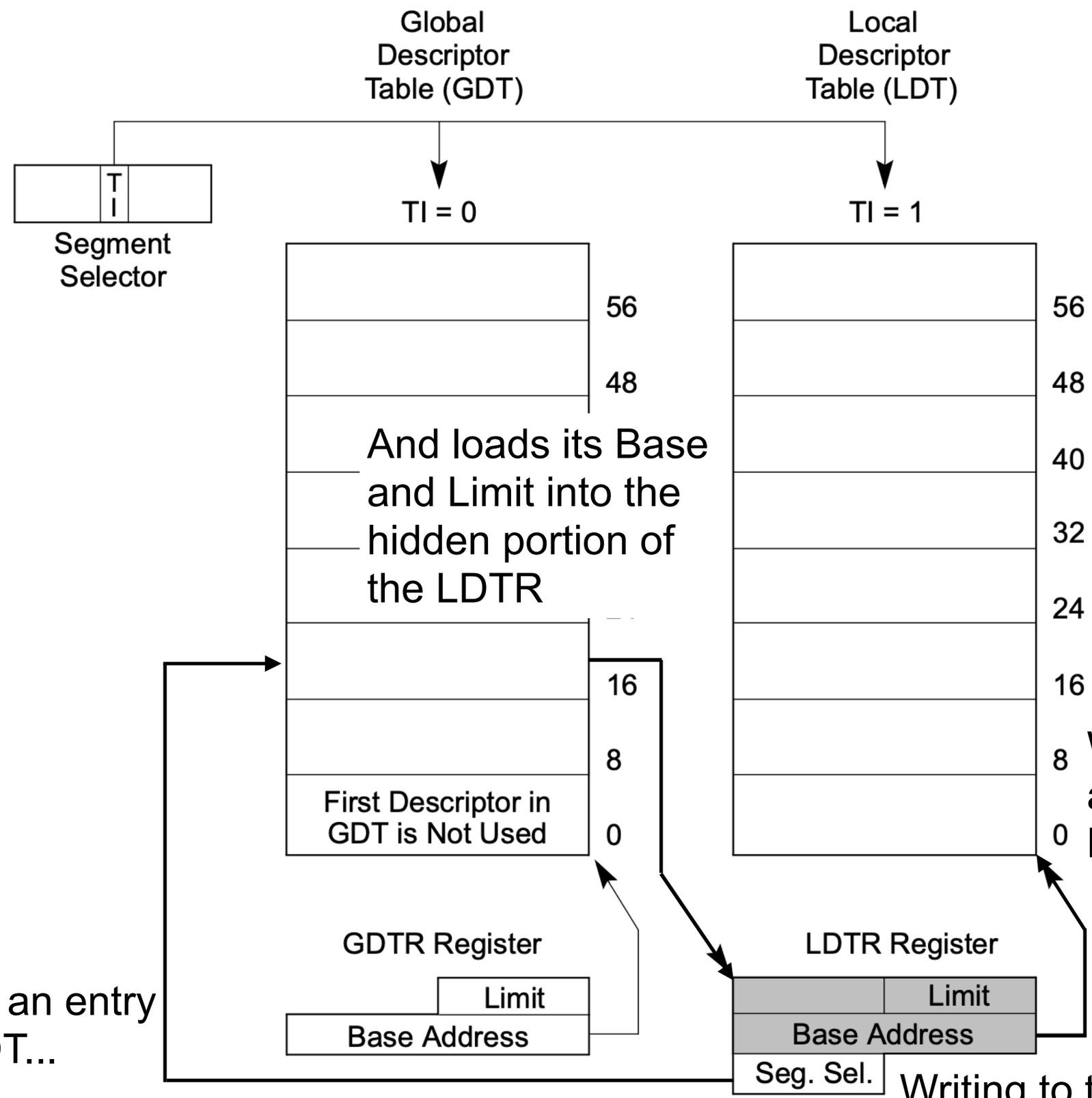


Figure 3-10. Global and Local Descriptor Tat

GDT & LDT



Looks up an entry in the GDT...

Writing to the LDTR segment selector...

Figure 3-10. Global and Local Descriptor Tat

What *Really* Happens With LDT...

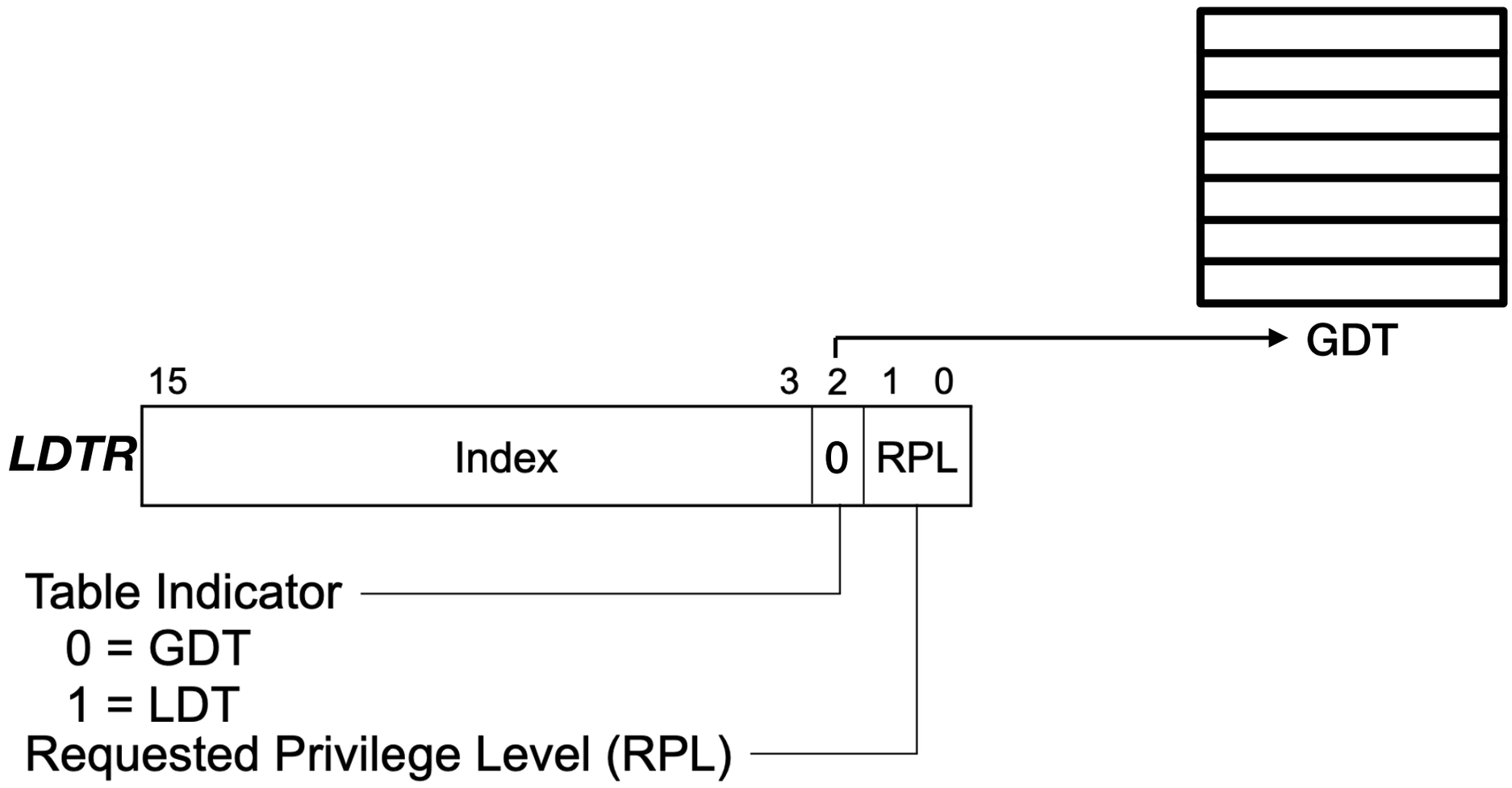


Figure 3-6. Segment Selector

What *Really* Happens With LDT...

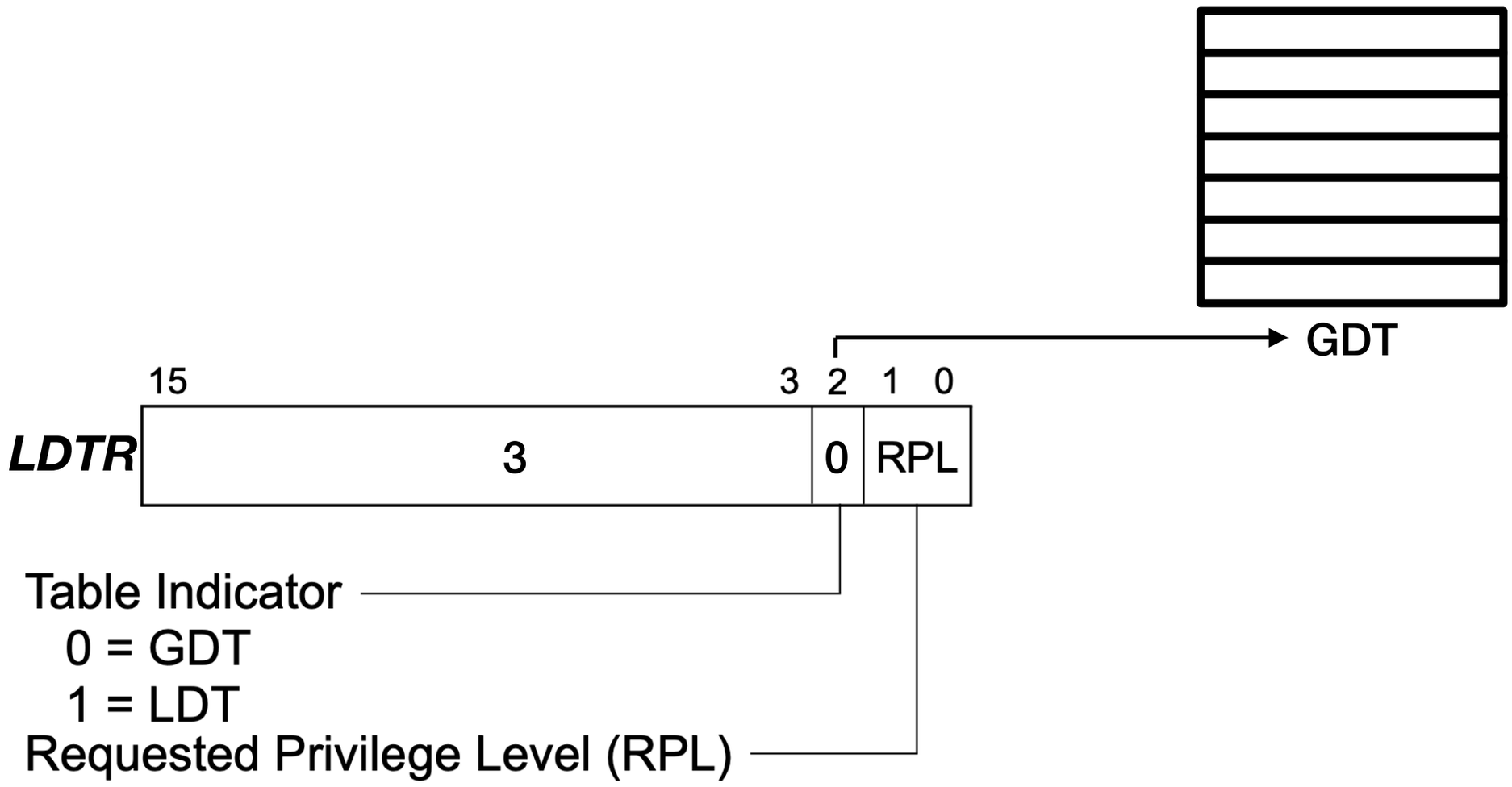


Figure 3-6. Segment Selector

What *Really* Happens With LDT...

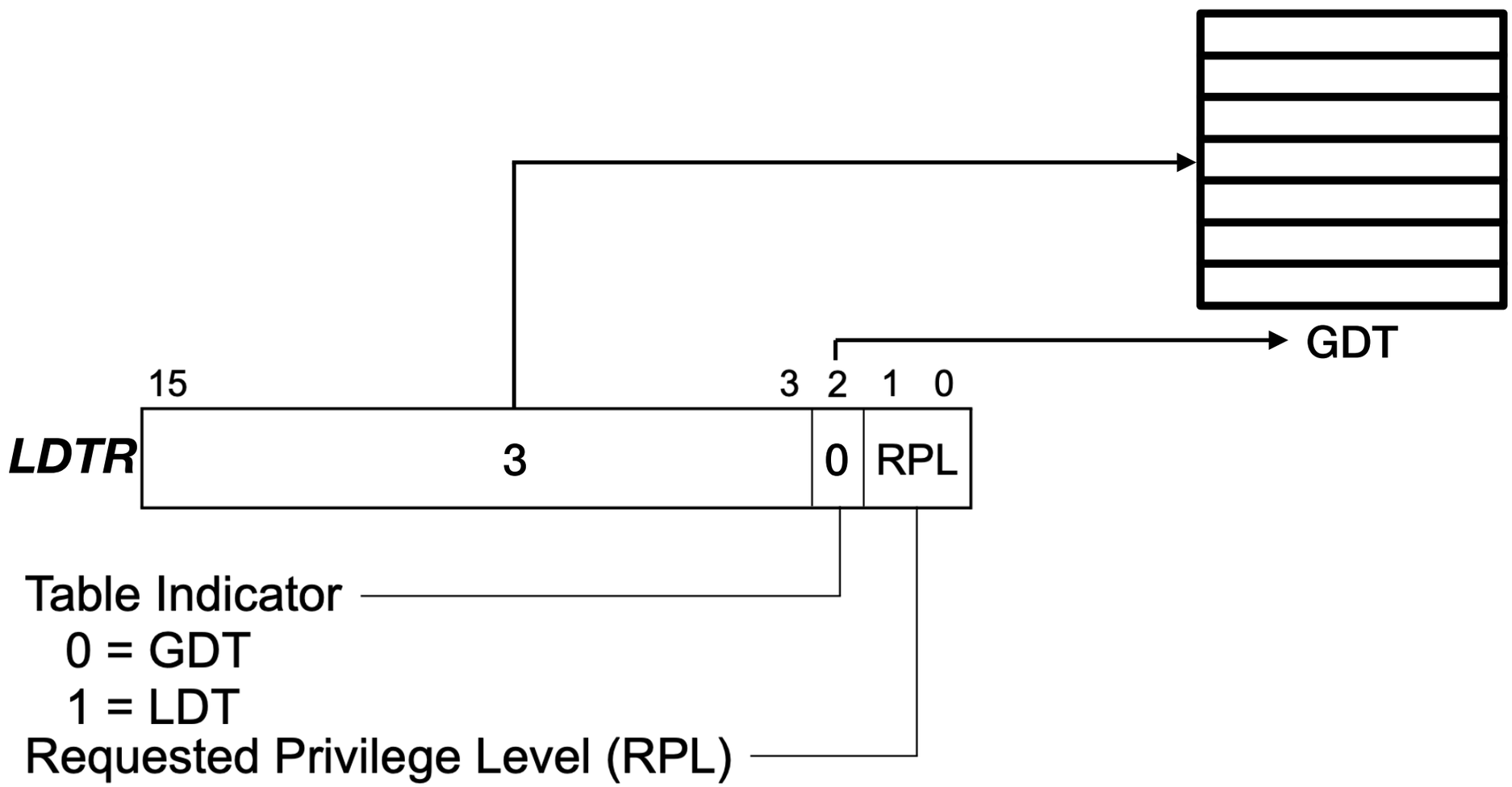


Figure 3-6. Segment Selector

What *Really* Happens With LDT...

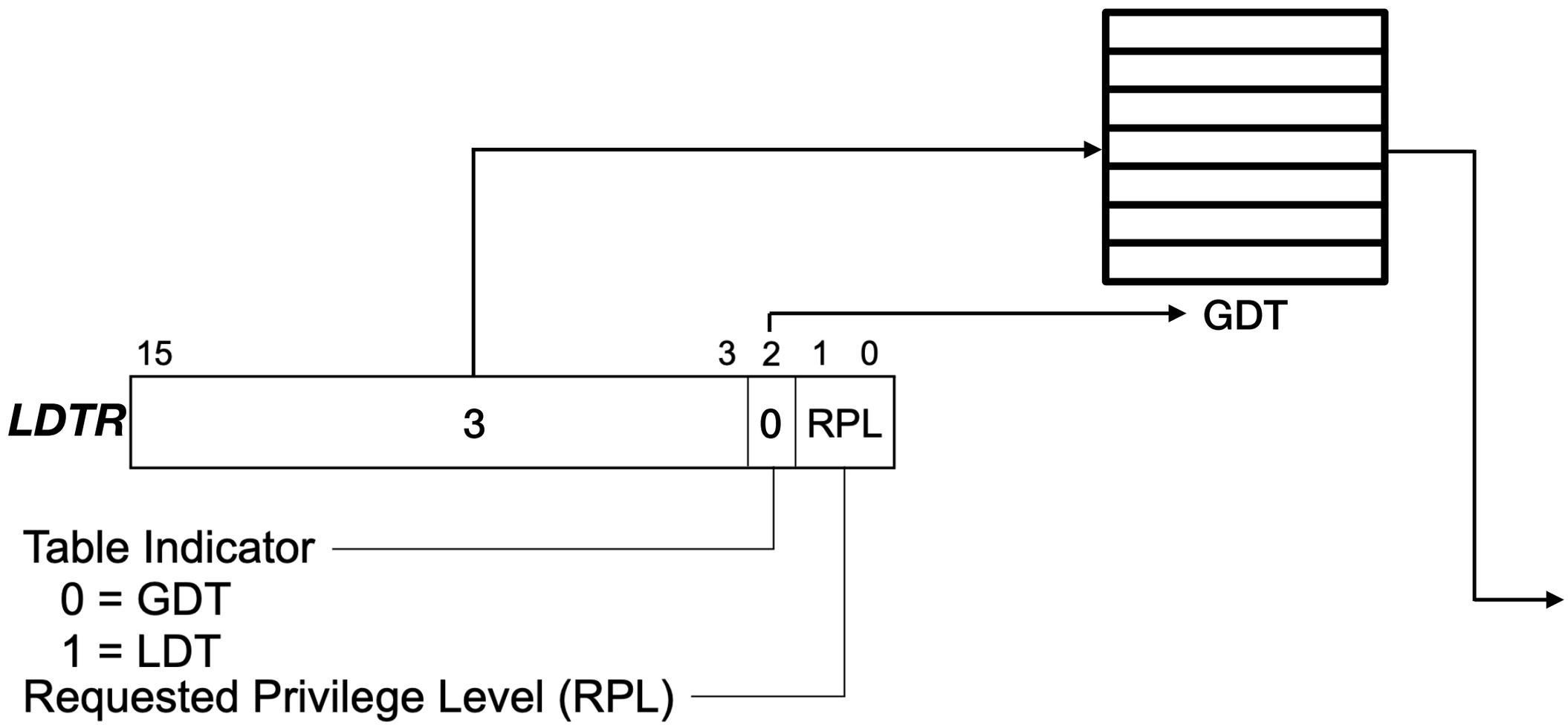


Figure 3-6. Segment Selector

What *Really* Happens With LDT...

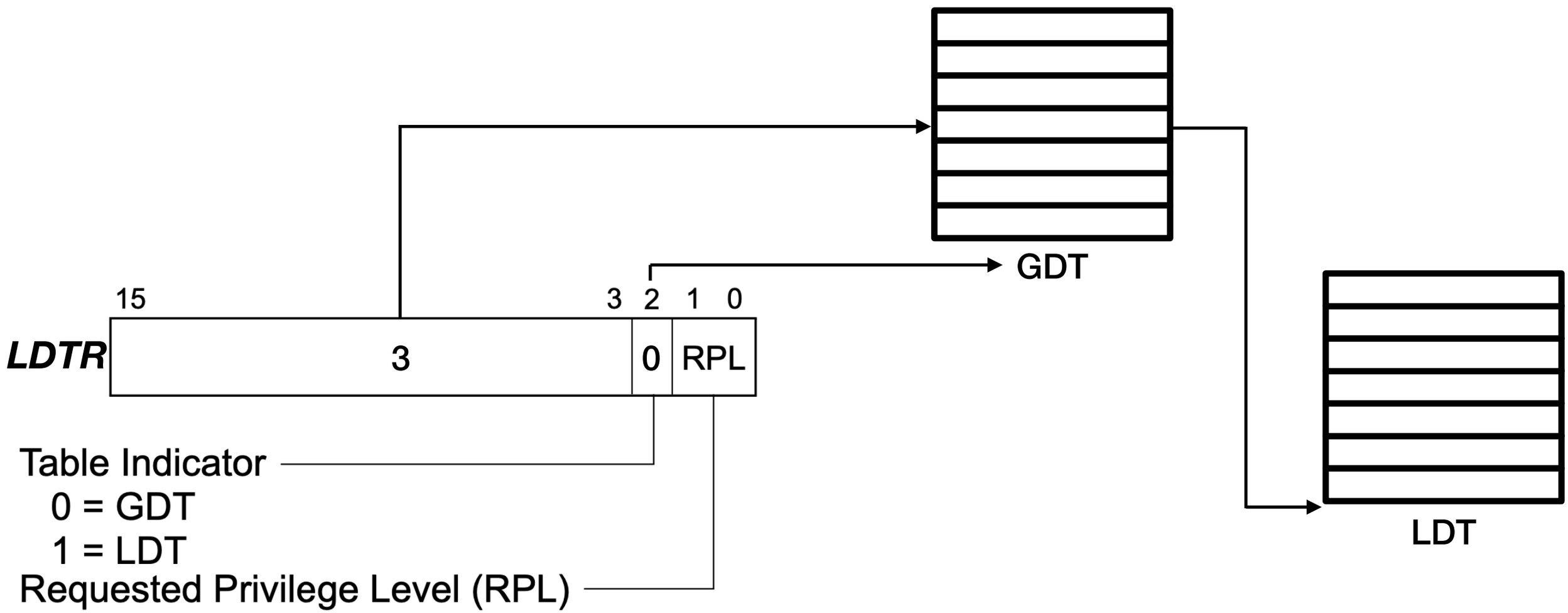


Figure 3-6. Segment Selector

What *Really* Happens With LDT...

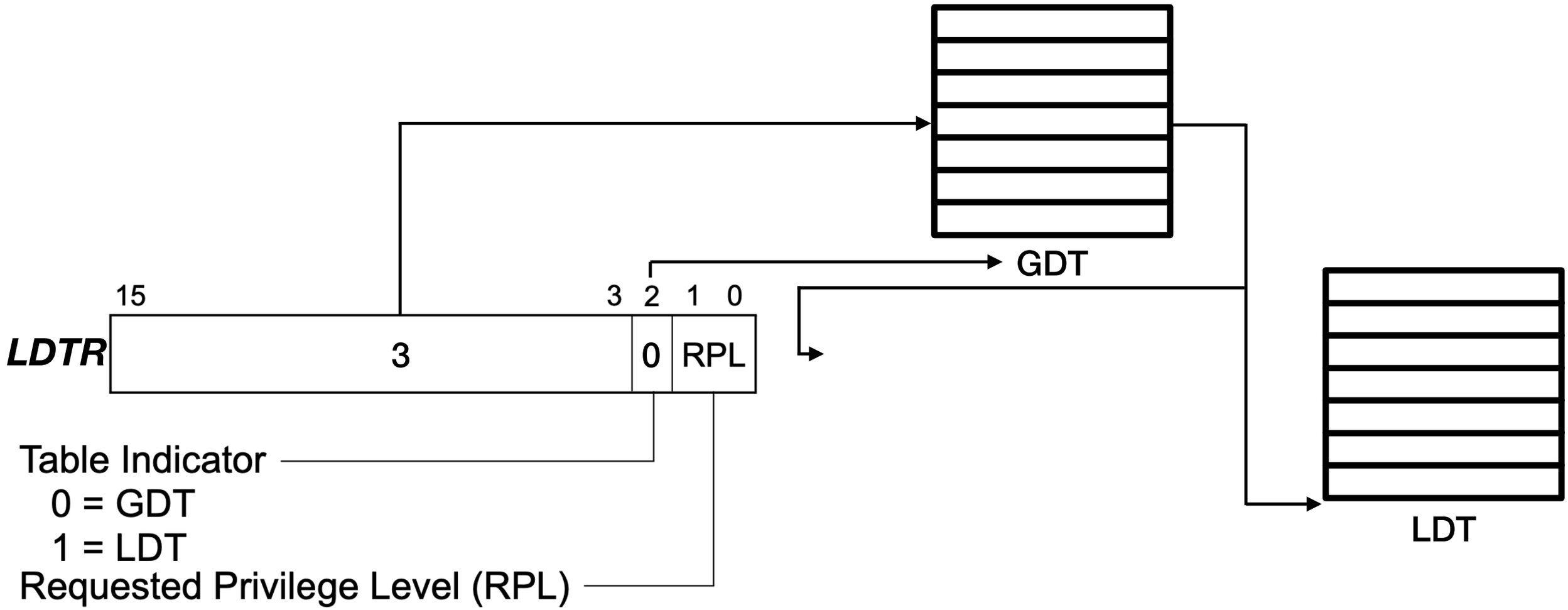


Figure 3-6. Segment Selector

What *Really* Happens With LDT...

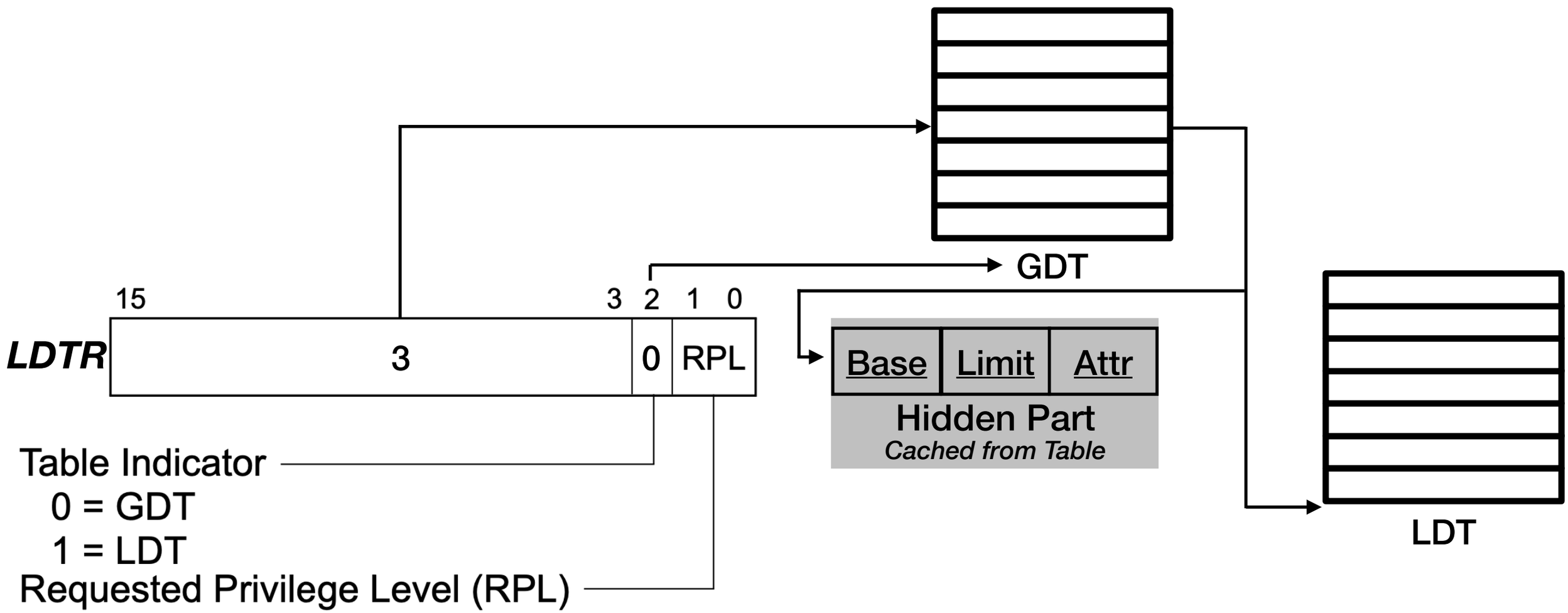


Figure 3-6. Segment Selector

What *Really* Happens With LDT...

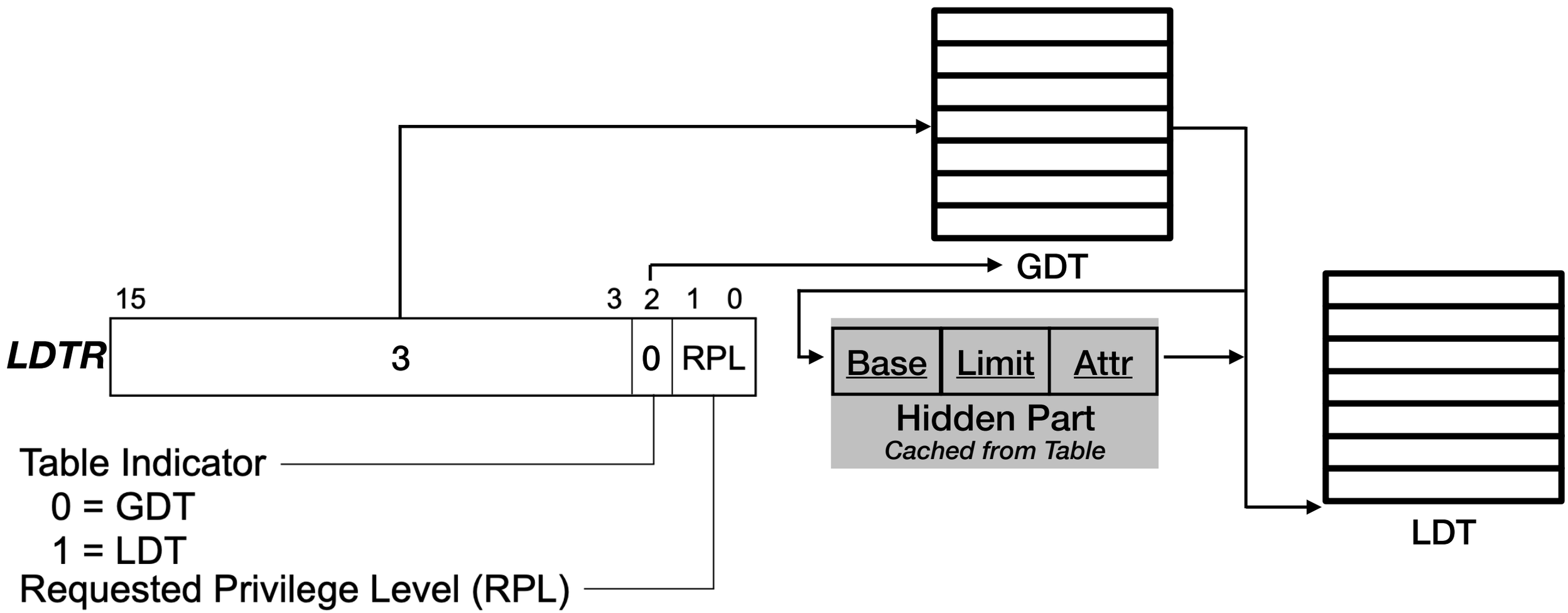


Figure 3-6. Segment Selector

GDT & LDT

In a register (i.e. CS, SS, etc) or LDTR

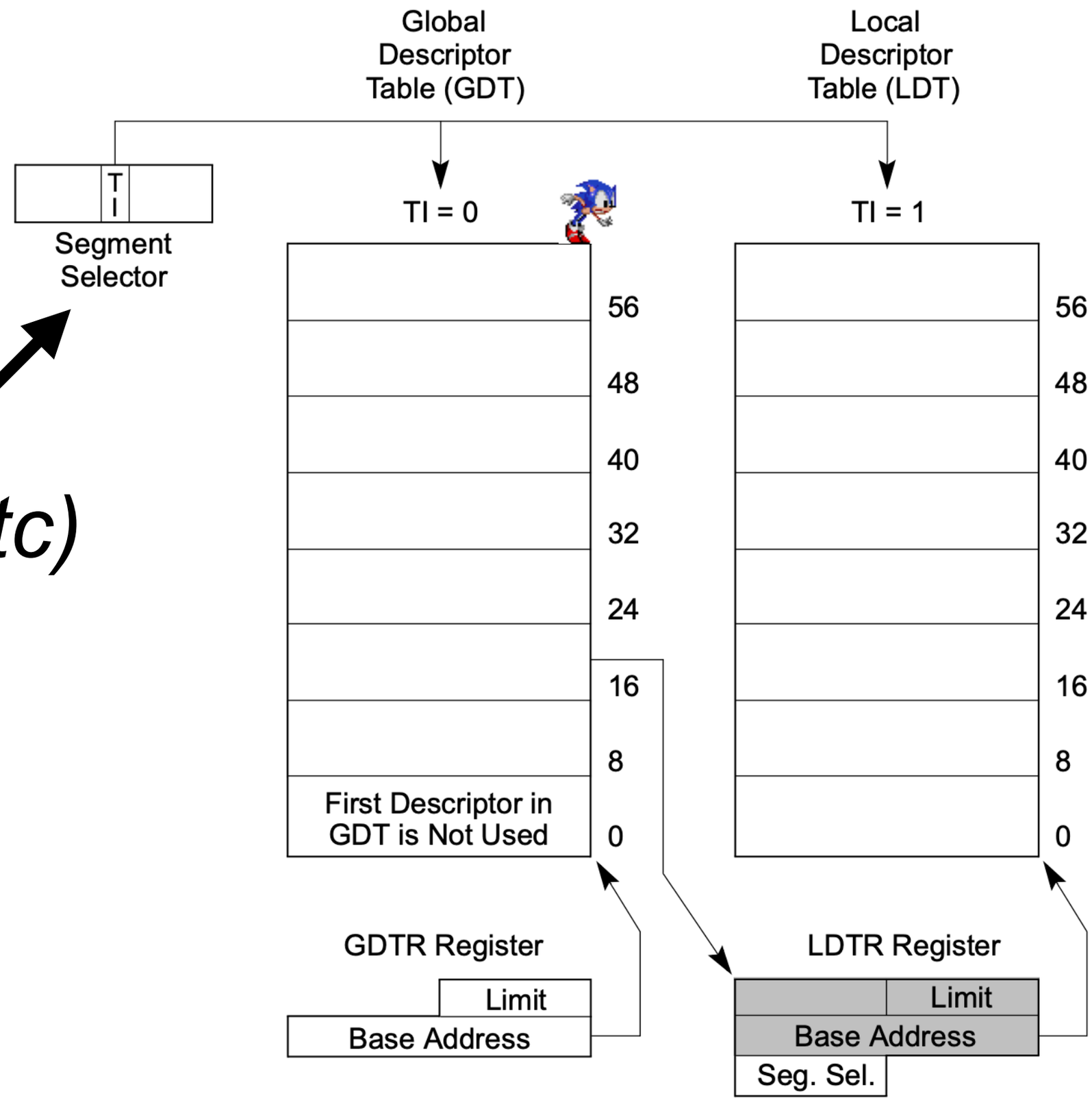


Figure 3-10. Global and Local Descriptor Tables

GDT & LDT

In a register (i.e. CS, SS, etc) or LDTR

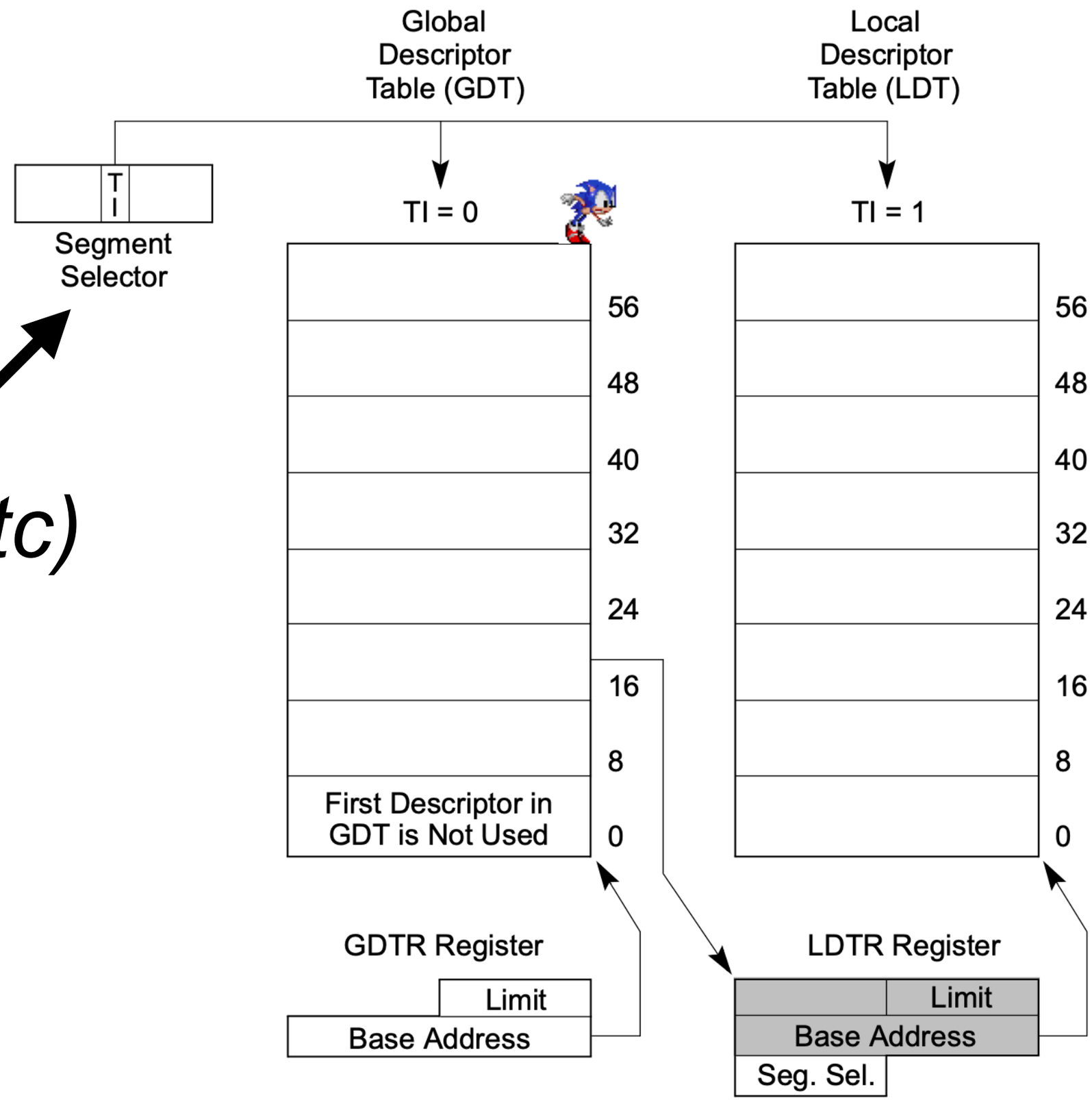
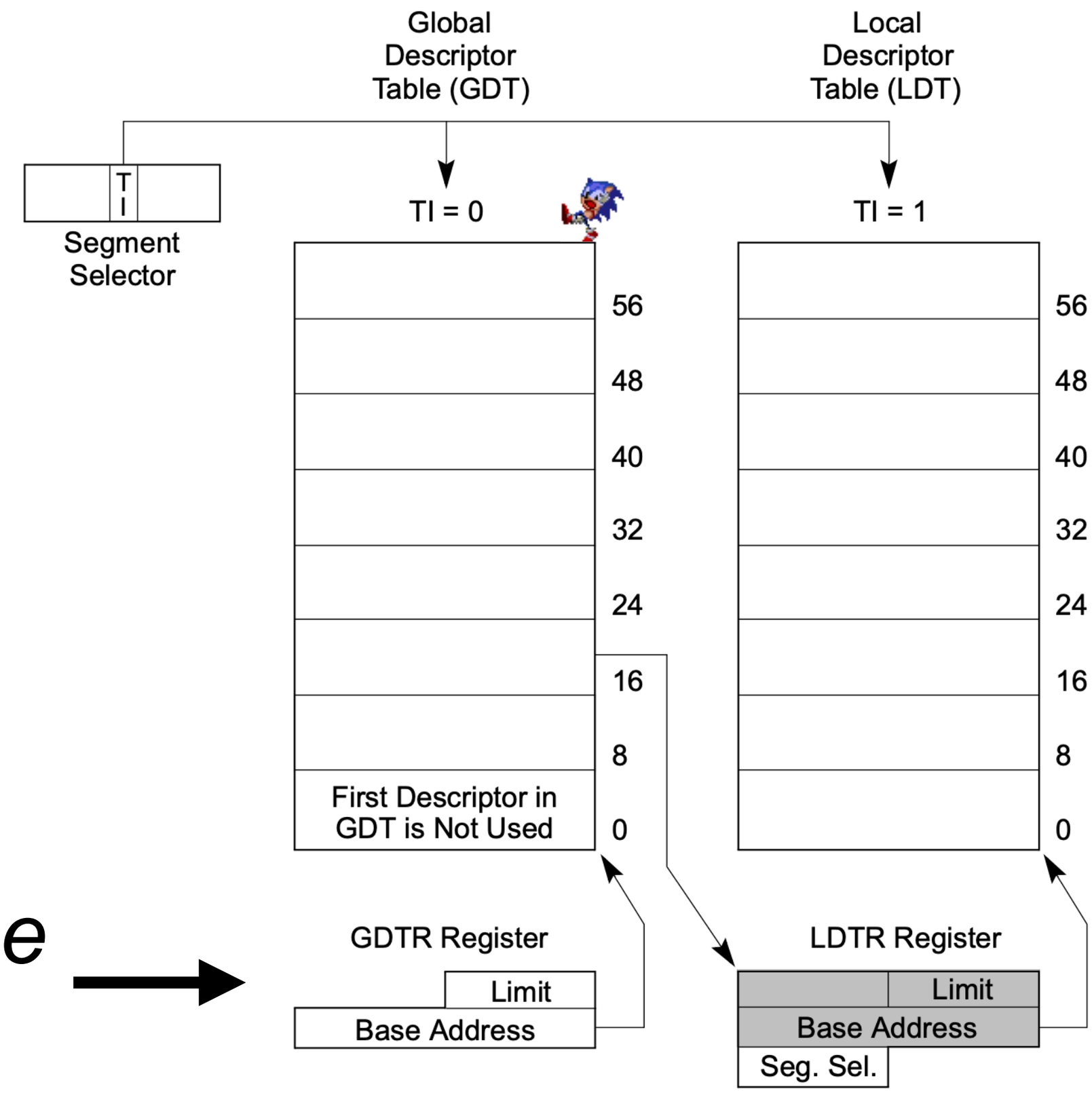


Figure 3-10. Global and Local Descriptor Tables

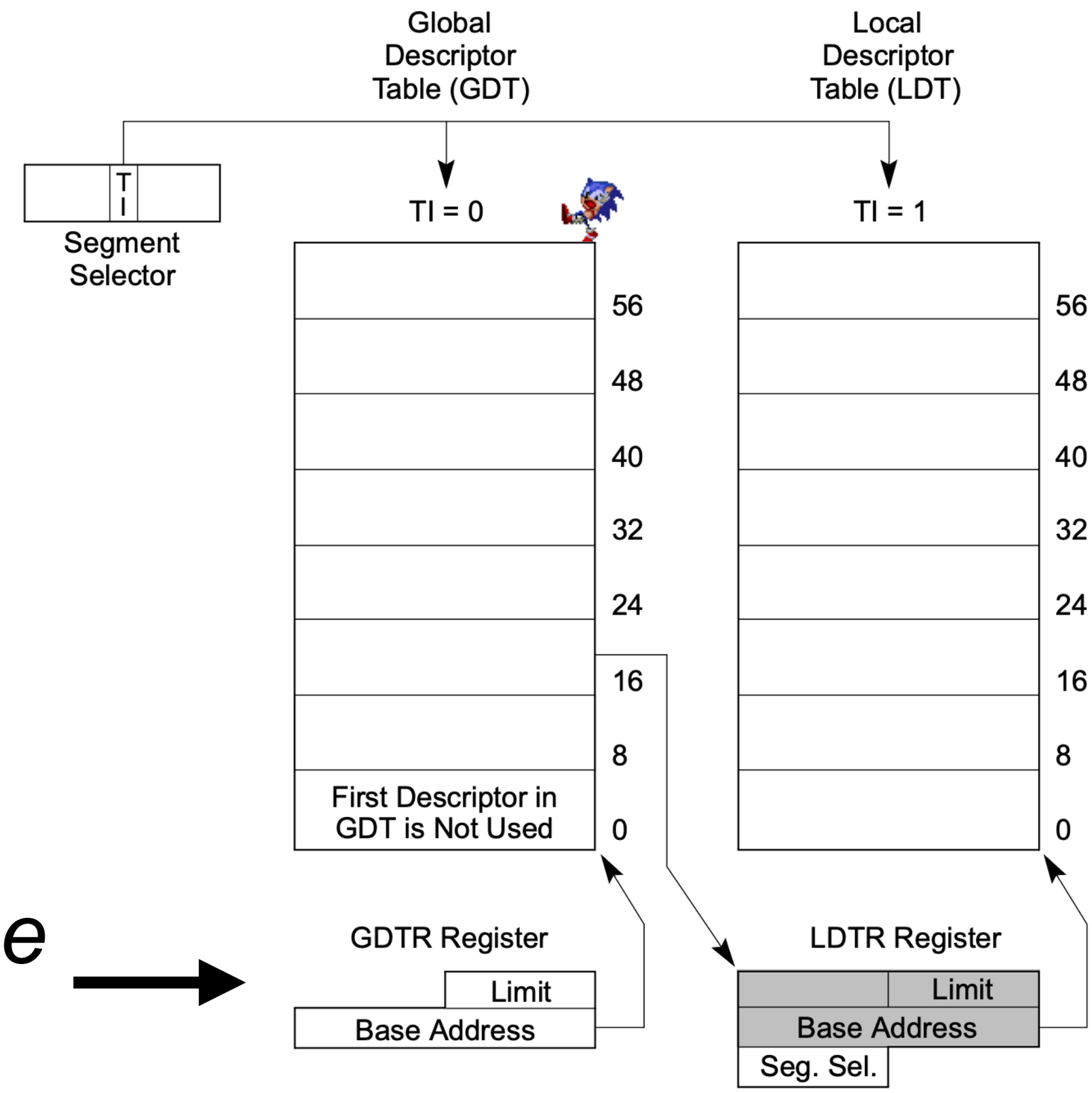
GDT & LDT



Special-purpose registers

Figure 3-10. Global and Local Descriptor Tables

GDT & LDT



Special-purpose registers

Figure 3-10. Global and Local Descriptor Tables

GDT & LDT

The tables themselves are <u>in RAM</u>

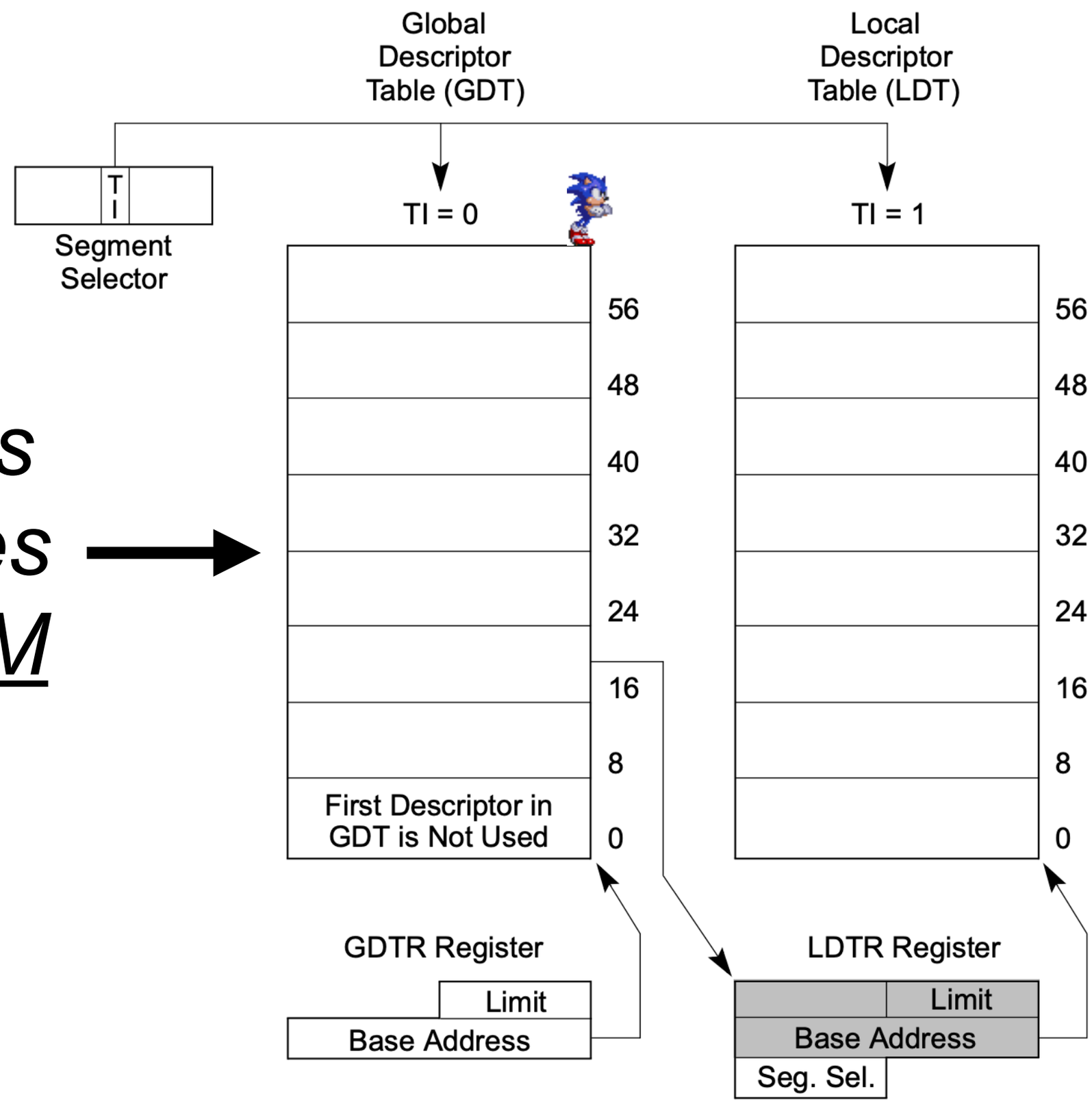


Figure 3-10. Global and Local Descriptor Tables

GDT & LDT

The tables themselves are <u>in RAM</u>

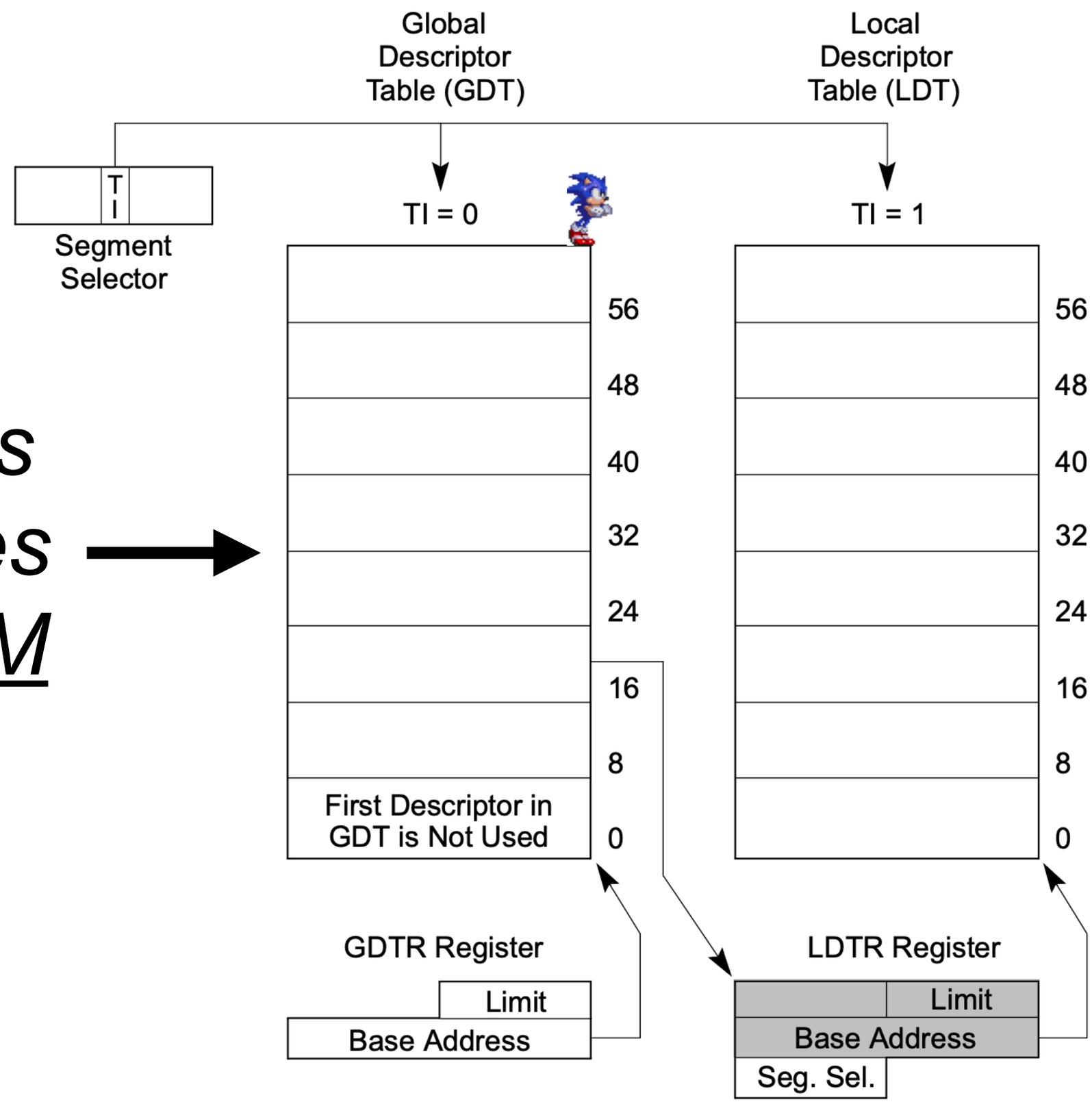


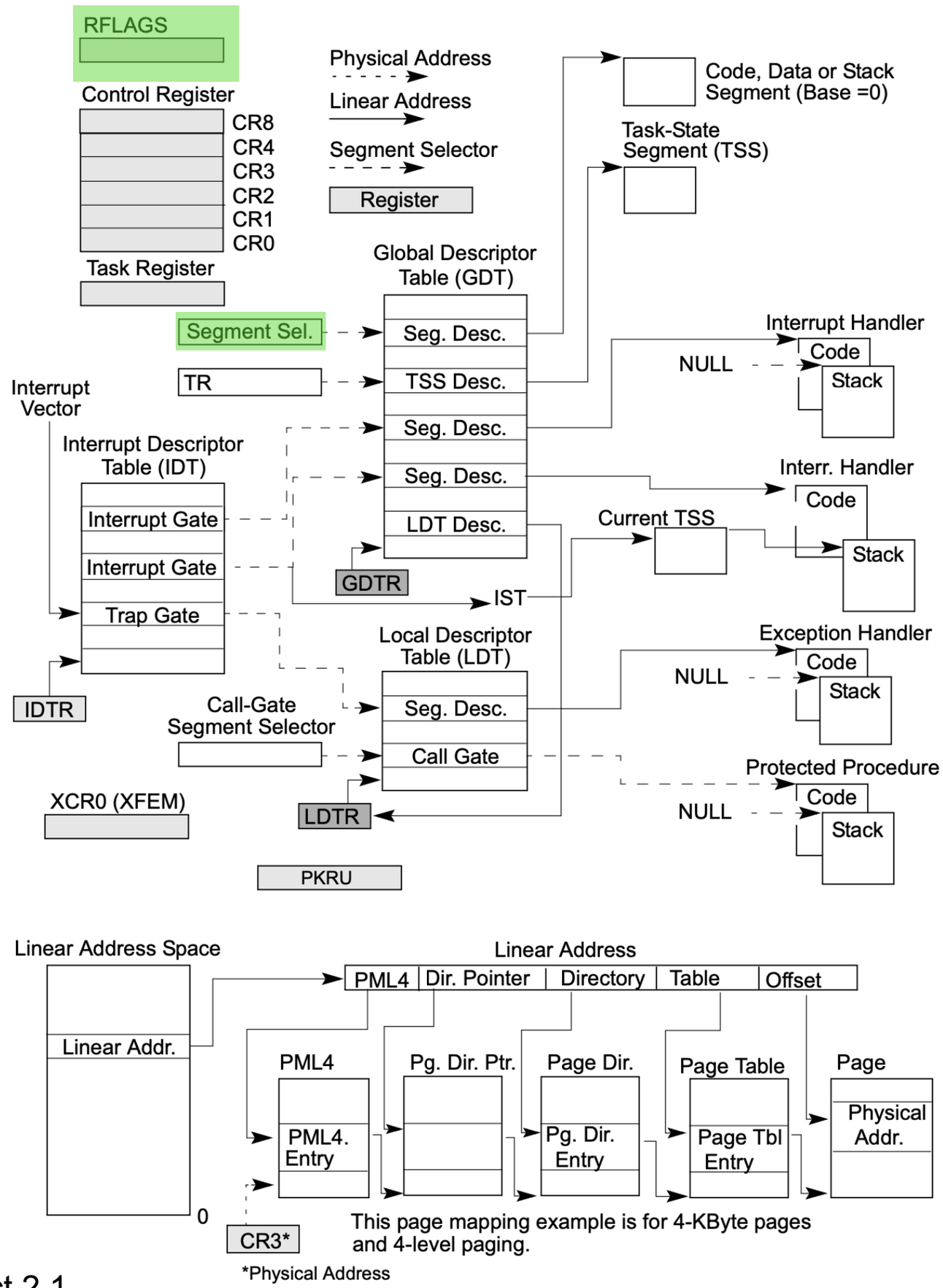
Figure 3-10. Global and Local Descriptor Tables

What's the Point of the LDT? 1

- The original point was to give different processes different views of memory via different segments
  - "In IA-32e mode, a segment descriptor table can contain up to 8192 (2^{13}) 8-byte descriptors." (Vol 3, Sect 3.5.2)
- These days, different views of memory for different processes is accomplished by *paging*, and no one uses the LDT for this

What's the Point of the LDT? 2

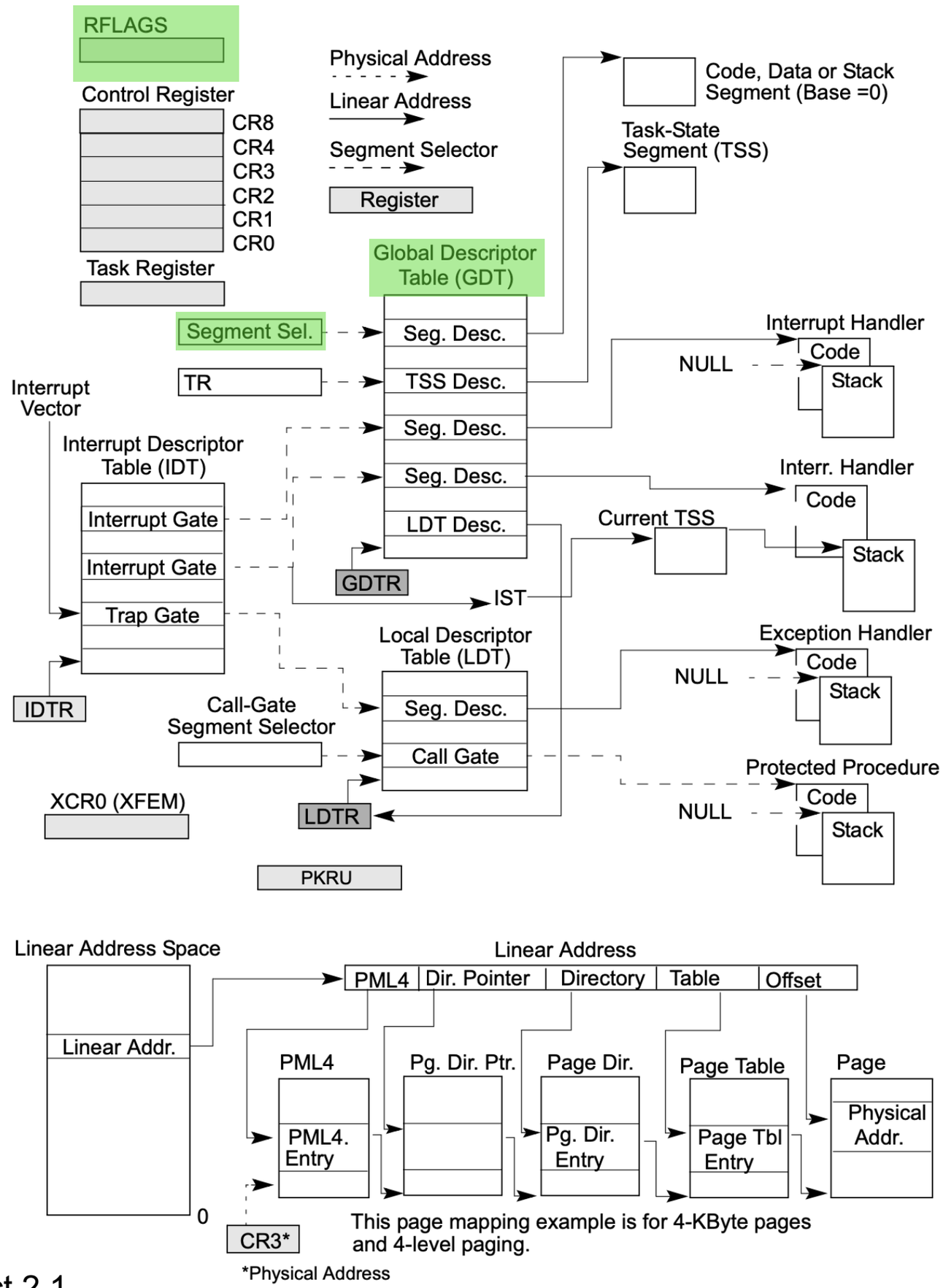
- Windows used the LDT for user-mode scheduling (UMS) but it was removed in a Windows 10 update as described here: https://www.alex-ionescu.com/?p=340



Vol 3, Sect 2.1

That's what you're going to learn! :D

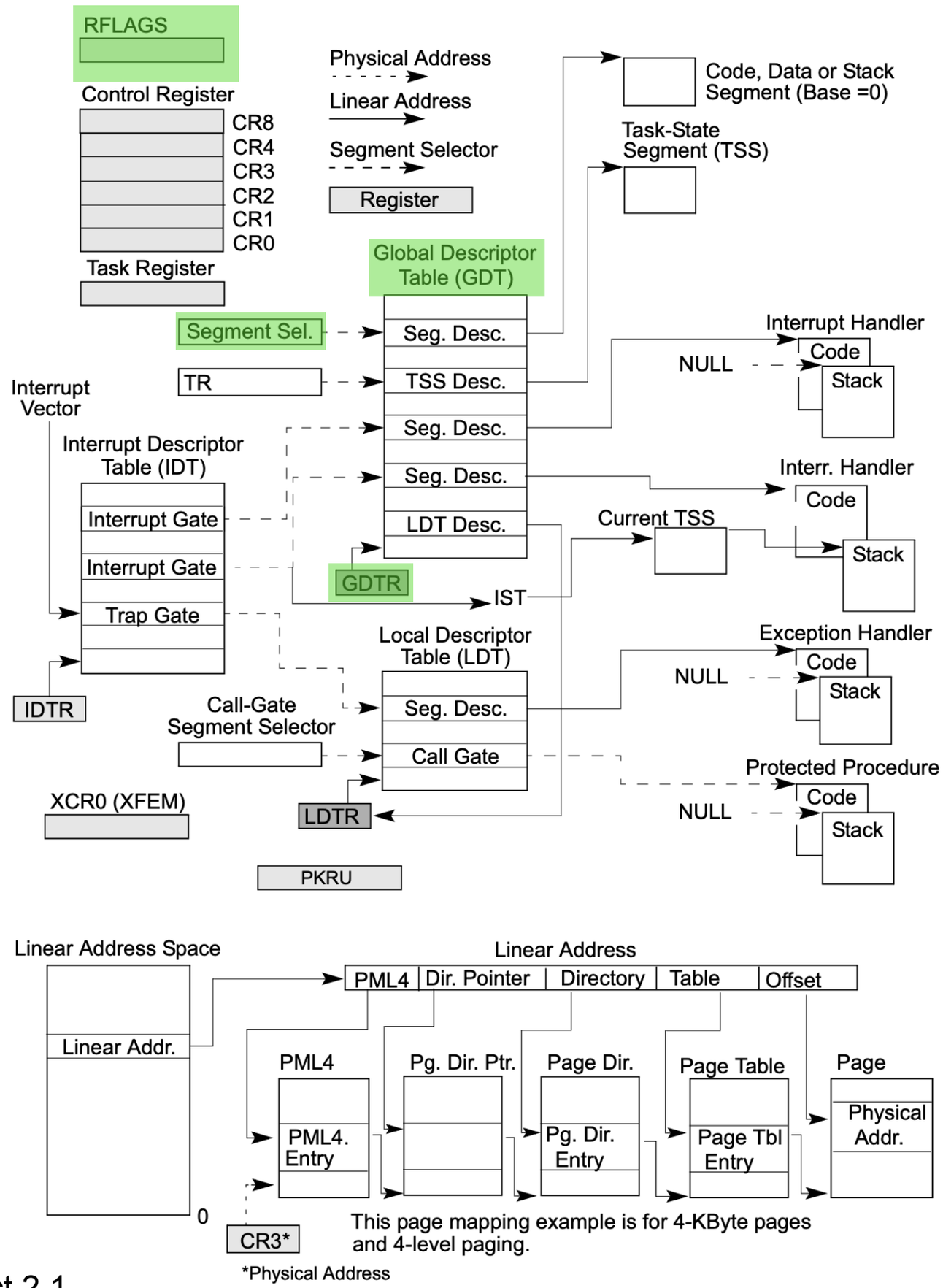
Figure 2-2. System-Level Registers and Data Structures in IA-32e Mode and 4-Level Paging



Vol 3, Sect 2.1

That's what you're going to learn! :D

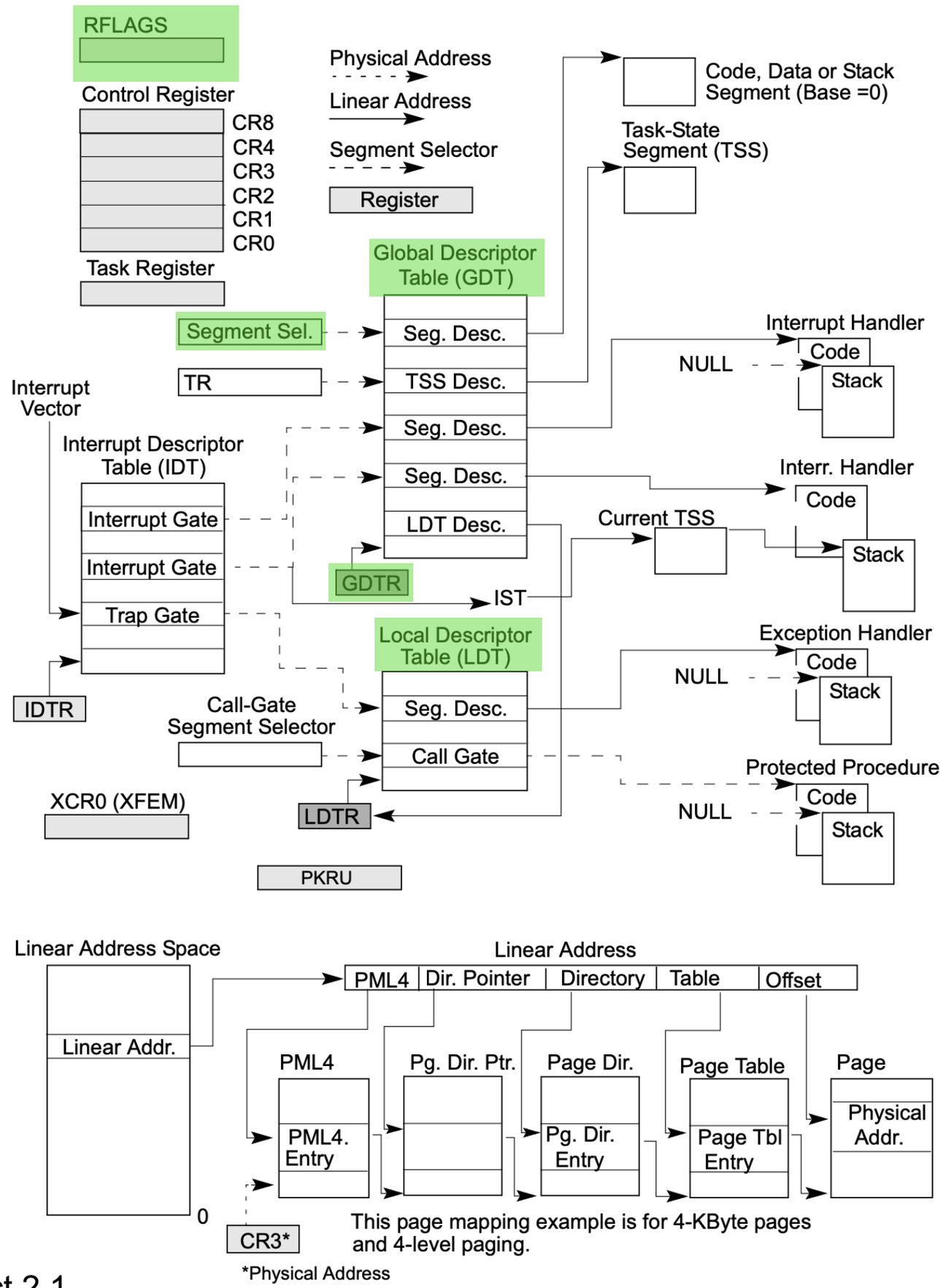
Figure 2-2. System-Level Registers and Data Structures in IA-32e Mode and 4-Level Paging



Vol 3, Sect 2.1

That's what you're going to learn! :D

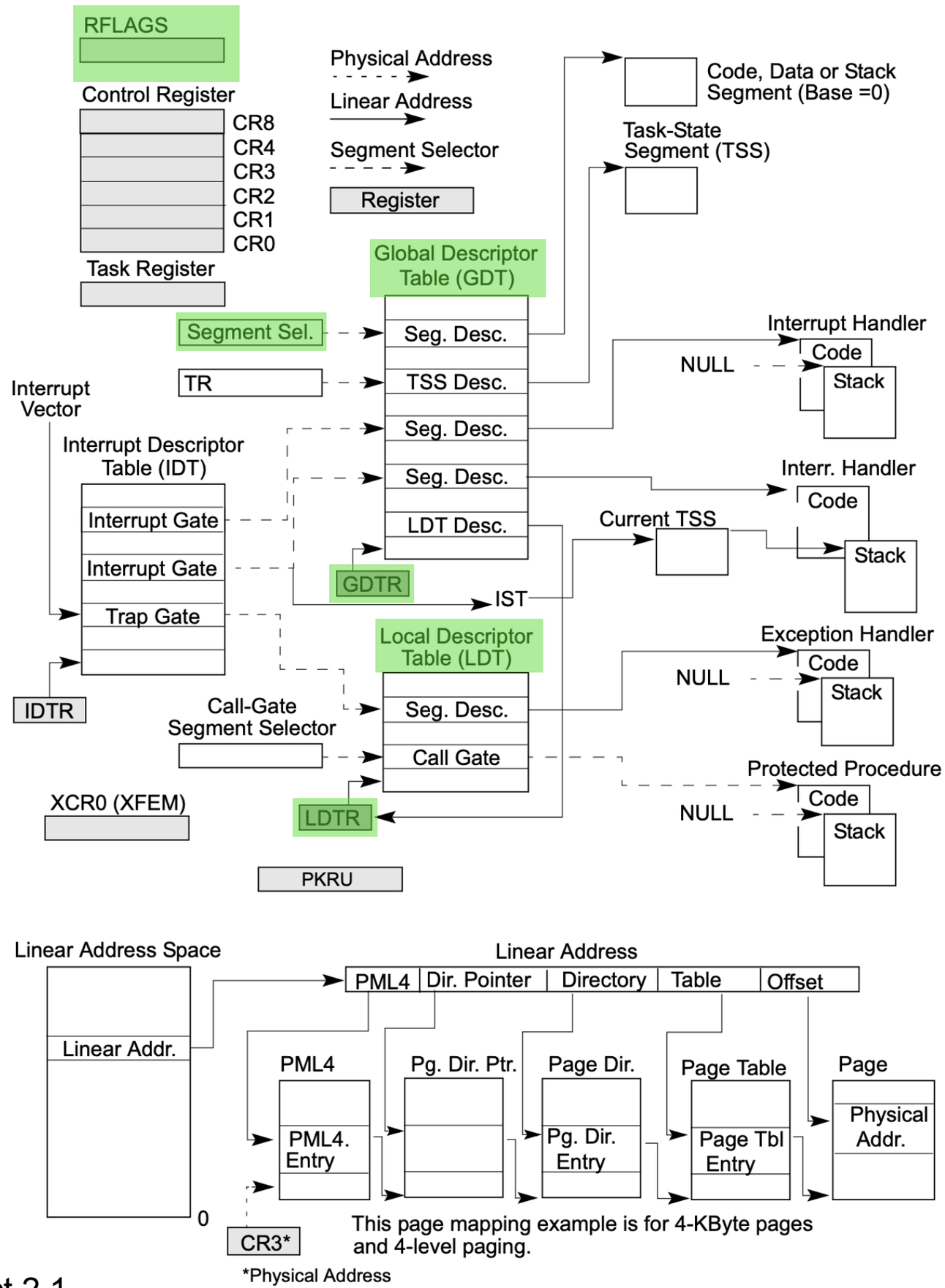
Figure 2-2. System-Level Registers and Data Structures in IA-32e Mode and 4-Level Paging



Vol 3, Sect 2.1

That's what you're going to learn! :D

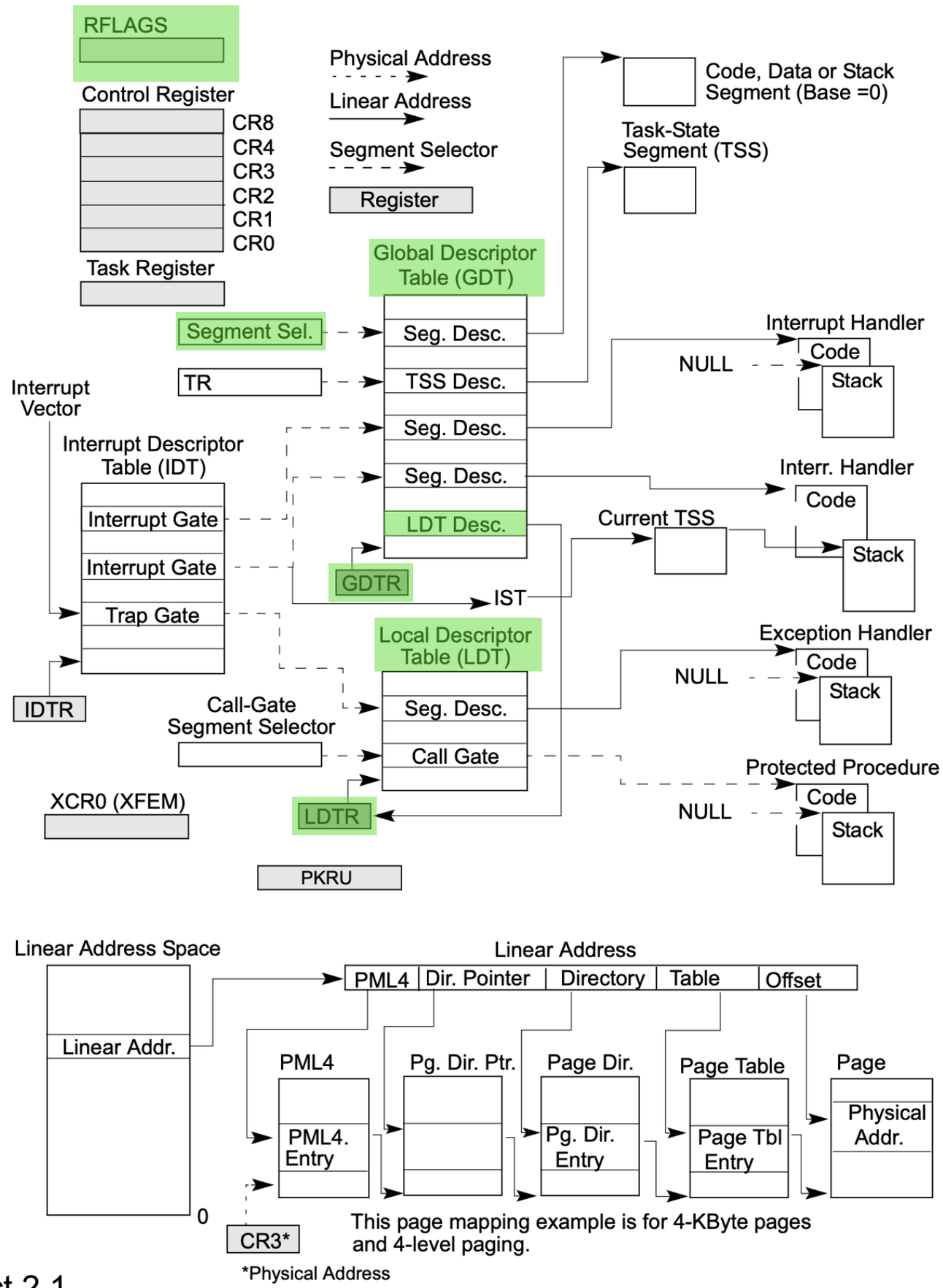
Figure 2-2. System-Level Registers and Data Structures in IA-32e Mode and 4-Level Paging



Vol 3, Sect 2.1

That's what you're going to learn! :D

Figure 2-2. System-Level Registers and Data Structures in IA-32e Mode and 4-Level Paging



Vol 3, Sect 2.1

That's what you're going to learn! :D

Figure 2-2. System-Level Registers and Data Structures in IA-32e Mode and 4-Level Paging